OPERATING SYSTEMS / STUDY EDITION

운영체제
핵심 요약 · 문제 · 정답 해설

19개 장의 요약 · 원문 번호 문제 103개 · 별도 측정 과제 2개 · 총 92쪽

> 각 장을 **요약 → 문제 원문 → 번호별 답·해설** 순서로 읽는다.
> 문제 원문을 먼저 풀고 뒤의 답과 풀이를 확인할 수 있게 구성했다.

이번 확장본에 추가한 것

각 장의 원본 문제 페이지, 계산·시뮬레이션의 검산표, 실행 순서·주소 변환 추적, 코드 과제의 예제, 실측 과제의 관측·해석 방법을 포함했다. 기존에 한 페이지에서 함께 요약했던 04·05, 13·14, 15·16장은 분리했다.

답의 세 가지 유형

유형	내용
계산·시뮬레이션	입력·기본값·시드를 명시하고 수치와 추적 결과를 검산
코드 과제	Linux/POSIX용 예시 코드와 정상·오류 동작 해석
실측·자유형 과제	정해진 단일 숫자 대신 재현 방법·예상 경향·결과 판별 기준

문제의 범위와 쪽수

첨부 PDF에 수록된 19개 장을 기준으로 했다. 02·23장에는 독립 숙제가 없어 요약만 수록했다. 13장의 영어 문제와 한국어 번역은 동일하므로 한국어 원문 한 번을 싣고 답 8개를 연결했다. 06장의 번호 없는 측정 숙제는 두 과제로 나눠 해설했다.

원본 PDF 쪽은 첨부 파일의 첫 페이지를 1로 센 위치다. 문제 페이지에 인쇄된 구판의 장·쪽번호를 보존했으며, 이 통합본의 위치는 바깥쪽 장 표시와 새 쪽번호로 확인한다.

자료: OS_merged_chapters_02-23.pdf, 280쪽 / 기존 20쪽 핵심 요약 / 교재 저자 OSTEP Homework 프로그램. 작성·검산 기준과 1차 자료 링크는 마지막 부록에 있다.

CONTENTS / 장별 바로 찾기

목차

장	주제	요약	문제	답·해설
02	운영체제 개요	3	-	-
04	프로세스	4	5	7
05	프로세스 API	9	10	12
06	제한적 직접 실행	17	18	20
07	CPU 스케줄링 기본	22	23	24
08	멀티 레벨 피드백 큐	26	27	28
09	비례 배분과 CFS	30	31	32
10	멀티프로세서 스케줄링	34	35	37
13	주소 공간	39	40	41
14	메모리 API	43	44	46
15	주소 변환의 원리	49	50	51
16	세그멘테이션	53	54	55
17	빈 공간 관리	57	58	59
18	페이징	62	63	65
19	TLB	67	68	71
20	더 작은 페이지 테이블	74	75	76
21	스왑 메커니즘	78	79	81
22	페이지 교체 정책	83	84	85
23	VAX/VMS·Linux 가상 메모리	88	-	-

장 번호는 첨부 북마크를 기준으로 통일했다. 03·11·12장은 첨부 파일에 없다. 문제 원문과 해설에 책갈피를 넣어 바로 이동할 수 있다.

부록: 핵심 공식과 검산 → 마지막 복습 → 답의 기준과 출처. 기존 요약의 공식·비교표도 보존했다.

02장 / 핵심 요약

운영체제 개요

CPU 가상화부터 가상 메모리·Linux 보충 자료까지

> **전체를 관통하는 질문**
> 여러 프로그램이 한 컴퓨터를 안전하고 효율적으로 공유하면서, 각자 CPU와 메모리를 가진 것처럼 실행되게 하려면 무엇이 필요한가?

운영체제의 세 가지 큰 주제

주제	핵심 역할	이 파일의 범위
가상화	물리 자원을 쓰기 쉬운 추상화로 제공	CPU·프로세스·메모리가 중심
병행성	실행 순서가 섞여도 공유 데이터를 정확히 관리	개요와 멀티 CPU 동기화
영속성	전원이 꺼져도 파일·데이터를 유지	개요에서만 소개

먼저 기억할 연결 관계

- **프로세스**는 실행 중인 프로그램이다. 스케줄러는 실행 대상을 고르고, 문맥 교환은 그 선택을 실제로 수행한다.
- **주소 공간**은 프로세스가 보는 메모리다. MMU와 페이지 테이블은 가상 주소를 물리 주소로 바꾸고 접근 권한을 검사한다.
- **TLB**는 주소 변환을 빠르게 하고, **스왑과 페이지 교체**는 RAM에 모든 페이지가 들어가지 않을 때를 처리한다.

설계 원칙

정책(policy)은 "누구를 실행할까, 무엇을 내보낼까"를 정한다. **메커니즘(mechanism)**은 "어떻게 전환·변환·전송할까"를 수행한다. 둘을 분리하면 정책을 바꿔도 공통 구현을 재사용할 수 있다.

목표는 편리한 추상화, 낮은 시간·공간 비용, 보호·격리, 신뢰성이다. 응답성·공정성·처리량, 시간·공간, 성능·보안 사이에는 절충이 있다.

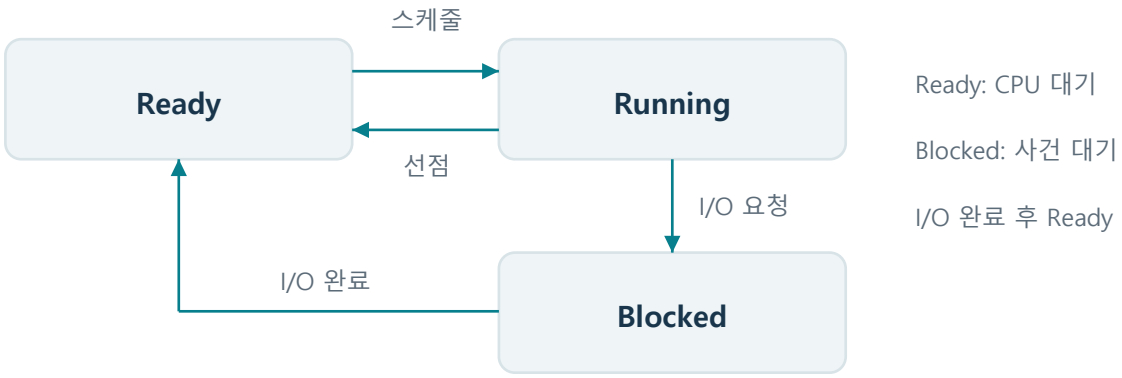
02장에는 독립 숙제·연습문제가 없다. 이 요약 뒤에는 04장 프로세스 요약과 해당 문제·정답 해설이 이어진다.

04장 / 핵심 요약

프로세스

> **프로세스 = 실행 중인 프로그램 + 실행 상태**
> 코드만 있는 파일과 달리, 주소 공간·레지스터·PC·열린 파일 등 현재 실행에 필요한 상태를 갖는다.

세 가지 상태와 전이



상태	기다리는 것
Running	현재 CPU에서 명령 실행
Ready	CPU 배정
Blocked	I/O 등 사건의 완료

운영체제가 관리하는 정보

PCB에는 PID, 상태, 저장된 레지스터·PC·스택 포인터, 주소 공간 정보, 열린 파일과 I/O 상태 등을 보관한다. 준비 작업과 대기 작업을 관리해 사건 완료 시 적절한 작업을 깨운다.

프로그램을 실행 상태로 만들기

코드·정적 데이터를 주소 공간에 배치하고 스택·힙·인자를 준비한다. 표준 입력·출력·오류의 파일 디스크립터도 설정한다. 필요한 페이지는 요구 페이징으로 실제 접근할 때까지 탑재를 미룰 수 있다.

I/O로 한 프로세스가 막힌 동안 다른 준비 작업을 실행하면 CPU 계산과 I/O를 겹칠 수 있다. I/O 완료는 먼저 Ready로 돌아가는 사건이며 즉시 실행 여부는 스케줄링 정책에 달려 있다.

원본 PDF 18-30쪽. 다음에 해당 장의 문제 원문과 번호별 답·해설을 수록했다.

숙제

process-run.py 프로그램은 프로세스가 실행되면서 변하는 프로세스의 상태를 추적할 수 있고, 프로세스가 CPU를 사용하는지(예, add 명령어 실행) 입출력을 하는지(예, 디스크에 요청을 보내고 완료되기를 기다린다)를 알아볼 수 있다. 상세한 사항은 README 파일을 확인하기 바란다.

문제

1. 다음과 같이 플래그를 지정하고 프로그램을 실행시키시오: `./process-run.py -l 5:100,5:100`. CPU 이용률은 얼마가 되어야 하는가(예, CPU가 사용 중인 시간의 퍼센트?) 그러한 이용률을 예측한 이유는 무엇인가? `-c` 플래그를 지정하여 예측이 맞는지 확인하시오. 100
2. 이제 다음과 같이 플래그를 지정하고 실행시키시오: `./process-run.py -l 4:100,1:0`. 이 플래그는 4개의 명령어를 실행하고 모두 CPU만 사용하는 하나의 프로세스와 오직 입출력을 요청하고 완료되기를 기다리는 하나의 프로세스를 명시한다. 두 프로세스가 모두 종료되는 데 얼마의 시간이 걸리는가? `-c` 플래그를 사용하여 예측한 것이 맞는지 확인하시오. 11틱
3. 옵션으로 지정된 프로세스의 순서를 바꾸시오: `./process-run.py -l 1:0,4:100`. 이제 어떤 결과가 나오는가? 실행 순서를 교환하는 것은 중요한가? 이유는 무엇인가? (언제나처럼 `-c` 플래그를 사용하여 예측이 맞는지 확인하시오.) 7틱
4. 자, 다른 플래그에 대해서도 알아보자. 중요한 플래그 중 하나는 `-S`로서 프로세스가 입출력을 요청했을 때 시스템이 어떻게 반응하는지를 결정한다. 이 플래그가 `SWITCH_ON_END`로 지정되면 시스템은 요청 프로세스가 입출력을 하는 동안 다른 프로세스로 전환하지 않고 대신 요청 프로세스가 종료될 때까지 기다린다. 입출력만 수행하는 프로세스와 CPU 작업만 하는 프로세스 두 개를 실행시키면 어떤 결과가 발생하는가?(`-l 1:0,4:200 -c -S SWITCH_ON_END`) 11틱
5. 이번에는 프로세스가 입출력을 기다릴 때마다 다른 프로세스로 전환하도록 플래그를 지정하여 같은 프로세스를 실행시켜 보자 (`-l 1:0,4:100 -c -S SWITCH_ON_IO`). 이제 어떤 결과가 발생하는가? `-c`를 사용하여 예측이 맞는지 확인하시오. 7틱
6. 또 다른 중요한 행동은 입출력이 완료되었을 때 무엇을 하느냐이다. `-I IO_RUN_LATER`가 지정되면 입출력이 완료되었을 때 입출력을 요청한 프로세스가 바로 실행될 필요가 없다. 완료 시점에 실행 중이던 프로세스가 계속 실행된다. 다음과 같은 조합의 프로세스를 실행시키면 무슨 결과가 나오는가? (`./process-run.py -l 3:0,5:100,5:100,5:100 -S SWITCH_ON_IO -I IO_RUN_LATER -c -p`) 시스템 자원은 효과적으로 활용되는가? 21틱
7. 같은 프로세스 조합을 실행시킬 때 `-I IO_RUN_IMMEDIATE`를 지정하고 실행시키시오. 이 플래그는 입출력이 완료되었을 때 요청 프로세스가 곧바로 실행되는 동작을

의미한다. 이 동작은 어떤 결과를 만들어 내는가? 방금 입출력을 완료한 프로세스를 다시 실행시키는 것이 좋은 생각일 수 있는 이유는 무엇인가?

8. 이제 다음과 같이 무작위로 생성된 프로세스를 실행시켜 보자. 예를 들면, `-s 1 -l 3:50,3:50`, `-s 2 -l 3:50,3:50`, `-s 3 -l 3:50,3:50`. 어떤 양상을 보일지 예측할 수 있는지 생각해 보시오. `-I IO_RUN_IMMEDIATE`를 지정했을 때와 `-I IO_RUN_LATER`를 지정했을 때 어떤 결과가 나오는가? `-S SWITCH_ON_IO` 대 `-S SWITCH_ON_END`의 경우에는 어떤 결과가 나오는가?

04장 / 정답 · 해설

프로세스

시뮬레이션 수치는 교재 저자의 공식 프로그램과 명시한 조건으로 검산했다. 미지정 옵션은 해당 버전의 기본값을 사용한다.

문제 1 · CPU만 사용하는 두 프로세스

답 총 10틱, CPU 이용률 **100%**다.

해설 각 프로세스가 CPU 명령 5개를 실행한다. I/O로 막히는 순간이 없어 CPU가 10틱 내내 일한다. 공식 프로그램의 -I 5:100,5:100 -c -p 결과로 확인했다.

문제 2 · CPU 작업을 먼저 실행한 경우

답 기본 I/O 길이 5에서 총 **11틱**, CPU 이용률 **54.55%**다.

해설 CPU 명령 4틱 뒤 I/O 요청 1틱, 대기 5틱, I/O 완료 명령(io_done) 1틱이 이어진다. 이 버전의 시뮬레이터는 요청과 완료 처리도 CPU 시간으로 세므로 CPU 사용은 6틱이다. 시스템의 실제 문맥 교환 시간 측정값을 뜻하는 것은 아니다.

문제 3 · I/O 작업을 먼저 실행한 경우

답 총 **7틱**, CPU 이용률 **85.71%**다.

해설 1틱에 I/O를 요청하고, 대기하는 동안 다른 프로세스의 CPU 명령 4개를 처리한다. 6틱에는 CPU가 쉬고 7틱에 I/O 완료 처리를 한다. 계산과 I/O가 겹쳐 2번의 11틱보다 짧다.

문제 4 · SWITCH_ON_END

답 총 **11틱**, CPU 이용률 **54.55%**다.

해설 I/O 프로세스가 끝날 때까지 전환하지 않으므로 I/O 대기 5틱 동안 CPU가 논다. 원문의 4:200은 퍼센트 표기로는 오기다. 이 프로그램은 200을 항상 CPU 명령을 고르는 값으로 처리하며 4:100으로 고쳐도 결과가 같다.

문제 5 · SWITCH_ON_IO

답 총 **7틱**, CPU 이용률 **85.71%**다.

해설 I/O가 시작되면 다른 준비 프로세스로 전환한다. I/O 프로세스를 앞에 둔 3번과 같은 상황이므로 대기와 계산을 겹칠 수 있다.

문제 6 · I/O 완료 후 나중에 재실행

답 IO_RUN_LATER: 총 **31틱**, CPU 이용률 **67.74%**다.

해설 I/O가 끝났어도 실행 중인 CPU 작업을 계속한다. 다음 I/O 요청이 늦어지고, CPU 작업들이 끝난 뒤 나머지 I/O만 기다리는 구간이 생긴다. CPU는 총 21틱, I/O 장치는 15틱 사용된다.

문제 7 · I/O 완료 후 즉시 재실행

답 IO_RUN_IMMEDIATE: 총 **21틱**, CPU 이용률 **100%**다.

해설 I/O 작업을 바로 실행해 다음 I/O를 빨리 요청하면, 그 대기 동안 CPU 작업을 처리할 수 있다. 이 워크로드에서는 I/O와 계산이 잘 겹쳐 CPU 유휴 구간을 없앤다. 모든 워크로드에서 항상 이 정책이 최적인 것은 아니다.

문제 8 · 시드와 I/O·전환 정책 비교

답 아래 표가 -I 3:50,3:50, 기본 I/O 길이 5의 검산 결과다.

해설 같은 시드라도 두 정책이 일정을 달리 만든다. SWITCH_ON_IO는 다른 작업으로 대기 시간을 덮는다. IMMEDIATE와 LATER가 같은 결과인 시드도 있으며, 시드 3에서는 즉시 재실행이 1틱을 절약한다. 확률 50%는 명령 생성 규칙이므로 실제 짧은 명령열의 CPU 비율이 정확히 50%일 필요는 없다.

시드	전환	완료 후	총 틱	CPU 이용률
1	IO	LATER	15	53.33%
1	IO	IMMEDIATE	15	53.33%
1	END	LATER	18	44.44%
1	END	IMMEDIATE	18	44.44%
2	IO	LATER	16	62.50%
2	IO	IMMEDIATE	16	62.50%
2	END	LATER	30	33.33%
2	END	IMMEDIATE	30	33.33%
3	IO	LATER	18	50.00%
3	IO	IMMEDIATE	17	52.94%
3	END	LATER	24	37.50%
3	END	IMMEDIATE	24	37.50%

05장 / 핵심 요약

프로세스 API

> Unix의 핵심 조합은 **fork → 환경 설정 → exec**다. 부모는 필요한 시점에 wait/waitpid로 자식 상태를 회수한다.

API	역할	기억할 점
fork	자식 생성	부모: 자식 PID / 자식:0 / 실패:-1
exec 계열	현재 프로그램 이미지 교체	성공 후 기존 코드로 복귀하지 않으며 PID 유지
wait/waitpid	자식 상태·종료 정보 회수	성공 반환은 PID, 종료 상태는 별도 매크로로 해석

셸과 입출력 연결

셸은 fork 후 자식에서 파일 디스크립터를 바꾸고 exec를 호출한다. stdout을 파일로 연결하면 리다이렉션, pipe의 쓰기·읽기 끝을 두 프로세스에 연결하면 파이프라인이 된다. dup2는 선택한 FD 번호에 연결을 복제한다.

부모와 자식의 관계

fork 후 실행 순서는 비결정적이다. 주소 공간은 논리적으로 분리되며 COW로 복사 비용을 늦출 수 있다. fork 전에 열린 FD는 동일한 열린 파일 설명을 공유하므로 오프셋도 공유한다.

wait는 여러 자식 중 회수 가능한 하나를, waitpid는 특정 PID 등 선택한 대상을 기다릴 수 있다. WNOHANG은 아직 변화가 없으면 0을 반환한다. 자식이 없는 wait는 ECHILD 오류다.

실수하기 쉬운 부분

printf의 사용자 공간 버퍼와 커널의 파일 오프셋은 다르다. 출력 순서에는 동기화가 필요하다. 파이프의 불필요한 끝을 모두 닫아야 EOF가 전달된다. 종료 코드와 wait 반환 PID도 구분한다.

원본 PDF 31-41쪽. 다음에 해당 장의 문제 원문과 번호별 답·해설을 수록했다.

숙제 (코드)

이 숙제에서는 방금 읽은 프로세스 관리 API와 친숙해 질 것이다. 걱정하지 마라. 들리는 것 보다는 훨씬 재미있으니까! 일반적으로 코드를 작성하기 위해 더 많은 시간을 투자하면 잘 하게 될 것이다[5]. 자 당장 시작하자.

여담: 코딩 숙제

코딩 숙제는 코드를 작성하여 실제 컴퓨터에서 실행시켜 봄으로써 현대 운영체제가 제공해야 하는 기본 API의 일부를 경험할 수 있는 작은 연습이다. 어쨌든 당신은 (아마도) 컴퓨터 과학자이니까 코딩을 좋아하겠지? 물론 진짜 전문가가 되려면 컴퓨터를 분해하는 시간보다는 더 많은 시간을 투자해야 한다. 사실 코드를 작성하여 실행시켜 동작을 이해할 수 있는 모든 구실을 찾아보아라. 시간을 투자해라. 그리고 될 수 있는 한 현명한 마스터가 되라.

문제

1. `fork()`를 호출하는 프로그램을 작성하라. `fork()`를 호출하기 전에 메인 프로세스는 변수에 접근하고 (예, x) 변수에 값을 지정하라 (예, 100). 자식 프로세스에서 그 변수의 값은 무엇인가? 부모와 자식이 변수 x를 변경한 후에 변수는 어떻게 변했는가?
2. `open()` 시스템 콜을 사용하여 파일을 여는 프로그램을 작성하고 새 프로세스를 생성하기 위하여 `fork()`를 호출하라. 자식과 부모가 `open()`에 의해 반환된 파일 디스크립터에 접근할 수 있는가? 부모와 자식 프로세스가 동시에 파일에 쓰기 작업을 할 수 있는가?
3. `fork()`를 사용하는 다른 프로그램을 작성하라. 자식 프로세스는 "hello"를 출력하고 부모 프로세스는 "goodbye"를 출력해야 한다. 항상 자식 프로세스가 먼저 출력하게 하라. 부모가 `wait()`를 호출하지 않고 할 수 있는가?
4. `fork()`를 호출하고 `/bin/ls`를 실행하기 위하여 `exec()` 계열의 함수를 호출하는 프로그램을 작성하라. `exec()`의 변형 `execl()`, `execle()`, `execlp()`, `execv()`, `execvp()`, `execve()` 모두를 사용할 수 있는지 시도해 보라. 기본적으로는 동일한 기능을 수행하는 시스템 콜에 여러 변형이 있는 이유를 생각해 보라.
5. `wait()`를 사용하여 자식 프로세스가 종료되기를 기다리는 프로그램을 작성하라. `wait()`가 반환하는 것은 무엇인가? 자식 프로세스가 `wait()`를 호출하면 어떤 결과가 발생하는가?

5) 코딩이 싫지만 컴퓨터 과학자가 되기 원한다면 (a) 컴퓨터 과학 이론 분야에서 잘 하거나 (b) 지금까지 들어 왔던 "컴퓨터 과학"이라는 것에 대해 다시 생각해 보아야 한다.

6. 위 문제에서 작성한 프로그램을 수정하여 `wait()` 대신에 `waitpid()`를 사용하라. 어떤 경우에 `waitpid()`를 사용하는 것이 좋은가?

7. 자식 프로세스를 생성하고 자식 프로세스가 표준 출력(`STDOUT_FILENO`)을 닫는 프로그램을 작성하라. 자식이 설명자를 닫은 후에 아무거나 출력하기 위하여 `printf()`를 호출하면 무슨 일이 생기는가?

8. 두 개의 자식 프로세스를 생성하고 `pipe()` 시스템 콜을 사용하여 한 자식의 표준 출력을 다른 자식의 입력으로 연결하는 프로그램을 작성하라.

dup() dup2() 를 알아야 한다

05장 / 정답 · 해설

프로세스 API

코드는 Linux/POSIX용 예시다. 이 Windows 작업 환경에서 Linux 하드웨어 수치를 실측하지 않았다. 실측 문항은 방법·해석·예상 경향을 답으로 제시한다.

문제 1 · fork 전후 변수 x

답 자식은 처음에 x=100을 본다. 이후 부모와 자식이 변경한 x는 서로 독립이다.

해설 fork가 주소 공간의 논리적 복사본을 만든다. COW로 물리 페이지를 잠시 공유해도 쓰기 때 분리된다. 아래에서 자식은 200, 부모는 300을 출력한다.

예시 코드

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void) {
    int x = 100;
    pid_t p = fork();
    if (p < 0) { perror("fork"); return 1; }
    if (p == 0) {
        printf("child initially: %d\n", x);
        x = 200;
        printf("child changed: %d\n", x);
        return 0;
    }
    x = 300;
    waitpid(p, NULL, 0);
    printf("parent changed: %d\n", x);
    return 0;
}
```

문제 2 · fork 전 open한 파일 디스크립터

답 부모와 자식 모두 파일을 쓸 수 있다. 같은 열린 파일 설명(open file description)의 오프셋과 상태 플래그를 공유한다.

해설 FD 번호는 각각의 프로세스 테이블에 있지만 동일한 커널 객체를 가리킨다. 각 write의 실행 순서는 정해지지 않는다. 레코드 순서를 보장하려면 동기화해야 한다. 표준 입출력의 사용자 공간 버퍼와 커널의 파일 오프셋을 혼동하지 않는다.

예시 코드

```
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void) {
    int fd = open("fork-file.txt", O_WRONLY|O_CREAT|O_TRUNC, 0600);
    if (fd < 0) { perror("open"); return 1; }
    pid_t p = fork();
    if (p < 0) { perror("fork"); close(fd); return 1; }
    const char *s = p == 0 ? "child\n" : "parent\n";
    size_t n = p == 0 ? 6 : 7;
    if (write(fd, s, n) != (ssize_t)n) perror("write");
    close(fd);
```

```
    if (p != 0) waitpid(p, NULL, 0);
    return 0;
}
```

문제 3 · wait 없이 hello를 먼저 출력

답 가능하다. 파이프 같은 명시적인 동기화 수단을 쓰면 된다. sleep만으로는 순서를 보장하지 못한다.

해설 자식이 hello를 출력하고 fflush한 뒤 파이프에 신호를 쓴다. 부모는 신호를 읽을 때까지 막혀 있으므로 goodbye가 나중에 출력된다. 이 예는 wait/waitpid를 호출하지 않고 출력 순서만 동기화한다.

예시 코드

```
#include <stdio.h>
#include <unistd.h>
int main(void) {
    int f[2]; char signal;
    if (pipe(f) < 0) { perror("pipe"); return 1; }
    pid_t p = fork();
    if (p < 0) { perror("fork"); return 1; }
    if (p == 0) {
        close(f[0]);
        puts("hello"); fflush(stdout);
        if (write(f[1], "x", 1) != 1) _exit(1);
        close(f[1]); _exit(0);
    }
    close(f[1]);
    if (read(f[0], &signal, 1) == 1) puts("goodbye");
    close(f[0]);
    return 0;
}
```

문제 4 · exec 계열 여섯 가지 변형

답 l은 가변 인자 목록, v는 argv 배열, p는 PATH 검색, e는 명시적 환경 배열을 뜻한다.

해설 호출자가 가진 인자의 형태와 실행 파일 검색·환경 설정 요구에 맞춘 인터페이스다. 성공하면 자식의 프로그램 이미지가 ls로 바뀌고 이전 코드로 돌아오지 않는다. 실행 파일 이름만 바뀌며 PID는 유지된다.

함수	인자	실행 파일	환경
execl	목록	경로	상속
execle	목록	경로	명시
execlp	목록	PATH	상속
execv	배열	경로	상속
execvp	배열	PATH	상속
execve	배열	경로	명시

예시 코드

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(int argc, char **argv) {
```

```
    if (argc != 2) return 1; /* choose 1..6 */
    pid_t p = fork();
    if (p < 0) { perror("fork"); return 1; }
    if (p == 0) {
        char *a[] = {"ls", "-l", NULL};
        char *e[] = {"PATH=/usr/bin:/bin", NULL};
        switch (atoi(argv[1])) {
        case 1: execl("/bin/ls","ls","-l",(char*)NULL); break;
        case 2: execle("/bin/ls","ls","-l",(char*)NULL,e); break;
        case 3: execlp("ls","ls","-l",(char*)NULL); break;
        case 4: execv("/bin/ls",a); break;
        case 5: execvp("ls",a); break;
        case 6: execve("/bin/ls",a,e); break;
        default: _exit(2);
        }
        perror("exec"); _exit(1);
    }
    waitpid(p, NULL, 0);
    return 0;
}
```

문제 5 · wait의 반환값과 자식의 wait

답 성공한 wait는 회수한 자식의 PID를 반환한다. 종료 코드는 status에서 매크로로 해석한다. 자식에게 자식이 없다면 -1과 ECHILD를 얻는다.

해설 PID와 종료 상태는 다른 값이다. 아래의 자식은 wait에 실패한 뒤 종료 코드 7로 끝나며 부모는 그 PID와 종료 코드 7을 확인한다. 일반적으로 wait는 여러 자식 중 먼저 회수 가능한 하나를 기다린다.

예시 코드

```
#include <stdio.h>
#include <errno.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void) {
    pid_t p = fork();
    if (p < 0) { perror("fork"); return 1; }
    if (p == 0) {
        pid_t r = wait(NULL);
        fprintf(stderr,"child wait=%ld ECHILD=%d\n",
                (long)r, errno == ECHILD);
        _exit(7);
    }
    int status; pid_t r;
    do { r = wait(&status); } while (r < 0 && errno == EINTR);
    if (r < 0) { perror("wait"); return 1; }
    printf("reaped=%ld exit=%d\n",(long)r,
           WIFEXITED(status) ? WEXITSTATUS(status) : -1);
    return 0;
}
```

문제 6 · waitpid를 쓰는 경우

답 특정 자식 PID를 기다리거나 WNOHANG으로 비차단 확인할 때 유용하다.

해설 waitpid(pid,&status;,0)은 지정한 자식, waitpid(-1,&status;,0)은 임의의 자식이다. WNOHANG에서 아직 회수할 상태가 없으면 0, 회수하면 PID, 오류면 -1이다. EINTR이면 재시도한다. 여러 자식을 가진 셸·서버에서 대상 선택이 필요하다.

5번 프로그램 부모 부분의 교체 코드

```
/* replace the parent wait loop in question 5 */
int status;
pid_t r = waitpid(p, &status, WNOHANG);
if (r == 0) {
    /* child still running: other work could be done here */
    do { r = waitpid(p, &status, 0); }
    while (r < 0 && errno == EINTR);
}
if (r < 0) perror("waitpid");
else if (WIFEXITED(status))
    printf("exit=%d\n", WEXITSTATUS(status));
```

문제 7 · stdout을 닫은 자식의 printf

답 표준 출력으로는 정상 출력되지 않는다. 실제 쓰기 시 닫힌 FD 때문에 EBADF가 발생할 수 있다.

해설 printf가 사용자 공간 버퍼에만 적었다면 바로 성공처럼 보일 수도 있다. fflush나 종료 때 버퍼를 내보내면서 실패가 드러난다. 따라서 printf의 결과뿐 아니라 fflush·ferror도 확인해야 한다. 오류 보고에는 열려 있는 stderr를 쓴다.

예시 코드

```
#include <stdio.h>
#include <errno.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void) {
    pid_t p = fork();
    if (p < 0) return 1;
    if (p == 0) {
        close(STDOUT_FILENO);
        errno = 0;
        int r = printf("hello\n");
        int f = fflush(stdout);
        fprintf(stderr,"printf=%d flush=%d error=%d errno=%d\n",
                r,f,ferror(stdout),errno);
        _exit(0);
    }
    waitpid(p,NULL,0);
    return 0;
}
```

문제 8 · 두 자식을 pipe로 연결

답 생산자의 stdout을 파이프 쓰기 끝에, 소비자의 stdin을 읽기 끝에 dup2로 연결한다.

해설 아래는 echo hello | wc -c에 해당하며 결과는 줄바꿈을 포함한 6이다. 두 자식과 부모 모두 불필요한 파이프 끝을 닫아야 한다. 부모가 쓰기 끝을 열어 두면 소비자가 EOF를 받지 못해 기다릴 수 있다.

예시 코드

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void) {
    int f[2];
    if (pipe(f) < 0) return 1;
    pid_t a = fork();
    if (a < 0) return 1;
    if (a == 0) {
        if (dup2(f[1],STDOUT_FILENO) < 0) _exit(1);
```

```
        close(f[0]); close(f[1]);
        execlp("echo","echo","hello",(char*)NULL);
        perror("echo"); _exit(1);
    }
    pid_t b = fork();
    if (b < 0) return 1;
    if (b == 0) {
        if (dup2(f[0],STDIN_FILENO) < 0) _exit(1);
        close(f[0]); close(f[1]);
        execlp("wc","wc","-c",(char*)NULL);
        perror("wc"); _exit(1);
    }
    close(f[0]); close(f[1]);
    waitpid(a,NULL,0); waitpid(b,NULL,0);
    return 0;
}
```

06장 / 핵심 요약

제한적 직접 실행

06장 | 일반 실행은 빠르게, 특권 작업과 CPU 회수는 통제

> **Limited Direct Execution, LDE**
> 프로그램은 CPU에서 직접 실행한다. 하드웨어의 모드·트랩·인터럽트 지원으로 운영체제가 제어권을 유지한다.

두 가지 해결 과제

과제	해결 장치	효과
위험한 연산 제한	사용자 모드 / 커널 모드	사용자 코드의 특권 명령 실행·커널 메모리 접근 제한
필요한 특권 서비스 제공	시스템 콜과 trap	정해진 진입점에서 인자·권한을 검사하고 요청 수행
CPU 제어권 회수	타이머 인터럽트	프로세스가 양보하지 않아도 운영체제가 선점 가능

시스템 콜의 경로

사용자 코드 → 라이브러리의 호출 래퍼 → 시스템 콜 번호·인자 준비 → trap → 커널 처리 → return-from-trap → 사용자 실행 재개.

트랩 테이블은 부팅 시 운영체제가 설정한다. 사용자 프로그램은 임의의 커널 주소로 진입할 수 없다. 하드웨어는 복귀에 필요한 상태를 저장하고 모드를 바꾼다.

문맥 교환의 경로

타이머 또는 다른 사건으로 커널 진입 → 스케줄러가 다음 프로세스 선택 → 현재 레지스터·커널 스택 문맥 저장 → 다음 문맥 복원 → 다음 프로세스의 사용자 코드로 복귀.

구분	의미
모드 전환	사용자 모드와 커널 모드 사이 이동. 같은 프로세스로 돌아올 수 있음
문맥 교환	실행 프로세스를 바꾸고 그 프로세스의 저장된 문맥으로 전환
협조 방식	프로세스의 시스템 콜·양보에 의존
비협조 방식	타이머 인터럽트로 제어권을 강제로 회수

인터럽트가 발생했다고 항상 다른 프로세스로 바뀌는 것은 아니다. 실행 대상을 고르는 정책과 전환을 수행하는 메커니즘을 구분한다.

숙제 (측정)

여담 : 측정 숙제

측정 숙제는 운영체제 또는 하드웨어 성능의 특정 측면을 측정하기 위해 실제 컴퓨터에서 실행시킬 코드를 작성하는 작은 연습이다. 이 숙제의 의도에는 운영체제를 직접 경험해 보자는 의미가 있다.

이 숙제에서 시스템 콜과 문맥 교환의 비용을 측정할 것이다. 시스템 콜의 비용을 측정하는 것은 상대적으로 쉽다. 예를 들어, 간단한 시스템 콜을(0 바이트 읽기 등) 반복적으로 호출하여 통틀어 걸린 시간을 측정한다. 이 시간을 반복 횟수로 나누면 시스템 콜의 비용을 추정할 수 있다.

한 가지 고려해야 할 사항은 타이머의 정확도와 정밀도이다. 사용할 수 있는 전형적인 타이머는 `gettimeofday()`이다. 자세한 사항에 대해서는 `man` 페이지를 참조하도록. `man` 페이지에서 볼 수 있는 정보는 `gettimeofday()`이 1970년 이후 경과한 시간을 마이크로초 단위로 반환한다는 것이다. 그러나 이 사실이 타이머가 마이크로초 수준의 정밀도를 가진다는 것을 의미하지는 않는다. 타이머가 얼마나 정밀한지를 알고 싶다면 `gettimeofday()`를 연속해서 호출하여 측정하라. 이 결과로부터 만족스러운 측정 결과를 얻기 위해서는 널 시스템 콜 테스트를 몇 번 반복해야 하는지 알 수 있을 것이다. `gettimeofday()`가 원하는 만큼 정밀도를 보이지 않는다면 x86 CPU에서 제공되는 `rdtsc` 명령어를 사용하는 것을 검토해 보아야 한다.

문맥 교환의 비용을 측정하는 것은 더 곤란하다. lmbench 벤치마크는 두 개의 프로세스를 하나의 CPU에서 실행시키고 둘 사이에 Unix 파이프를 설정하여 문맥 교환 비용을 측정한다. 파이프는 Unix 시스템에서 프로세스끼리 통신할 수 있는 여러 방법 중의 하나이다. 첫 번째 프로세스는 첫 번째 파이프에 데이터를 쓰고, 두 번째 파이프로부터 데이터가 전달되기를 기다린다. 첫 번째 프로세스가 두 번째 파이프로부터 읽을 무언가를 기다린다는 사실을 알게 되면 운영체제는 첫 번째 프로세스를 봉쇄 상태로 만들고 다른 프로세스로 전환한다. 이 프로세스는 첫 번째 파이프로부터 데이터를 읽고 두 번째 파이프에 데이터를 쓴다. 두 번째 프로세스가 첫 번째 파이프로부터 다시 데이터를 읽으려고 하면 봉쇄 상태로 들어간다. 이런 식으로 주고 받는 통신 사이클이 계속된다. 이러한 통신 비용을 반복적으로 측정하여 lmbench는 문맥 교환의 비용에 대해 훌륭한 추정치를 구할 수 있다. 이 숙제에서 파이프를 사용하거나 Unix 소켓과 같은 다른 통신 기법을 사용하여 유사한 상황을 만들어 볼 수 있다.

문맥 교환의 비용을 측정하는 데 있어 어려움은 하나 이상의 CPU를 가진 시스템에서 발생한다. 그런 시스템에서 해야만 하는 일은 문맥 교환되는 프로세스들이 같은 프로세서 상에 위치하는 것을 보장하는 것이다. 다행히도 대부분의 운영체제는 프로세스를 특정 프로세서에 묶어두는 시스템 콜을 가진다. 예를 들어, Linux에서는 `sched_setaffinity()` 시스템 콜이 기대하는 일을 한다. 두 프로세스를 같은 프로세

서에 존재하도록 보장하면 같은 프로세서에 존재하는 한 프로세스를 중단시키고 다른 프로세스를 복원하는 운영체제의 비용을 확실하게 측정할 수 있다.

06장 / 정답 · 해설

제한적 직접 실행

코드는 Linux/POSIX용 예시다. 이 Windows 작업 환경에서 Linux 하드웨어 수치를 실측하지 않았다. 실측 문항은 방법·해석·예상 경향을 답으로 제시한다.

측정 과제 1 · 측정 과제 1: 시스템 콜 비용과 타이머

답 고정된 정답 시간은 없다. 평균 비용은 (반복 호출의 총 시간 - 루프 기준 비용)/호출 횟수로 추정한다.

해설 gettimeofday의 출력 단위가 μs라고 해서 정확도·분해능도 1 μs인 것은 아니다. 연속 읽기의 차이, 측정 오버헤드, 전체 측정 시간이 충분한지 확인한다. 아래는 Linux에서 read 시스템 콜을 직접 반복한다. 실측을 수행한 수치가 아닌 재현 코드다. CLOCK_MONOTONIC은 벽시계 보정에 덜 영향을 받는다.

syscall-cost.c: gcc -O2 syscall-cost.c -o syscall-cost

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/syscall.h>
static double ns(void) {
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC,&t);
    return t.tv_sec*1e9+t.tv_nsec;
}
int main(int argc,char **argv) {
    long n = argc > 1 ? atol(argv[1]) : 1000000;
    if (n <= 0) return 1;
    int fd = open("/dev/null",O_RDONLY);
    if (fd < 0) return 1;
    char b; volatile long sink = 0;
    double t = ns();
    for (long i=0;i<n;i++) sink += 0;
    double baseline = ns()-t;
    t=ns();
    for (long i=0;i<n;i++) {
        if (syscall(SYS_read,fd,&b,0) < 0) return 1;
        sink += 0;
    }
    double elapsed=ns()-t;
    printf("raw_ns=%.2f estimated_ns=%.2f\n",
           elapsed/n,(elapsed-baseline)/n);
    close(fd); return 0;
}
```

측정 과제 2 · 측정 과제 2: 문맥 교환 비용

답 두 프로세스를 같은 CPU에 고정하고 파이프 왕복을 반복한다. 왕복 평균을 2로 나누면 단방향 전환과 IPC를 포함한 비용 추정치다.

해설 한 왕복에는 보통 부모→자식→부모 전환이 있다. read/write와 파이프 전달 비용도 들어가므로 순수한 레지스터 교환 비용이라고 단정하지 않는다. 여러 CPU에 흩어지면 두 프로세스가 동시에 실행돼 같은 CPU의 문맥 교환을 측정하지 못한다. 워밍업 후 여러 번 측정해 중앙값과 변동폭을 보고한다.

switch-cost.c: gcc -O2 switch-cost.c -o switch-cost

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>
#include <sched.h>
#include <sys/wait.h>
static double ns(void) {
    struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t);
    return t.tv_sec*1e9+t.tv_nsec;
}
int main(int argc,char **argv) {
    int n=argc>1 ? atoi(argv[1]) : 100000;
    if(n<=0) return 1;
    cpu_set_t set; CPU_ZERO(&set);
    int cpu=sched_getcpu(); if(cpu<0) return 1;
    CPU_SET(cpu,&set);
    if(sched_setaffinity(0,sizeof(set),&set)<0) return 1;
    int a[2],b[2]; char v='x';
    if(pipe(a)<0 || pipe(b)<0) return 1;
    pid_t p=fork(); if(p<0) return 1;
    if(p==0) {
        close(a[1]); close(b[0]);
        while(read(a[0],&v,1)==1)
            if(write(b[1],&v,1)!=1) _exit(1);
        _exit(0);
    }
    close(a[0]); close(b[1]);
    double t=0;
    for(int i=0;i<n+1000;i++) {
        if(i==1000) t=ns();
        if(write(a[1],&v,1)!=1 || read(b[0],&v,1)!=1) return 1;
    }
    double elapsed=ns()-t;
    printf("round_trip_ns=%.2f one_way_including_IPC_ns=%.2f\n",
           elapsed/n,elapsed/(2*n));
    close(a[1]); close(b[0]); waitpid(p,NULL,0);
    return 0;
}
```

07장 / 핵심 요약

CPU 스케줄링 기본

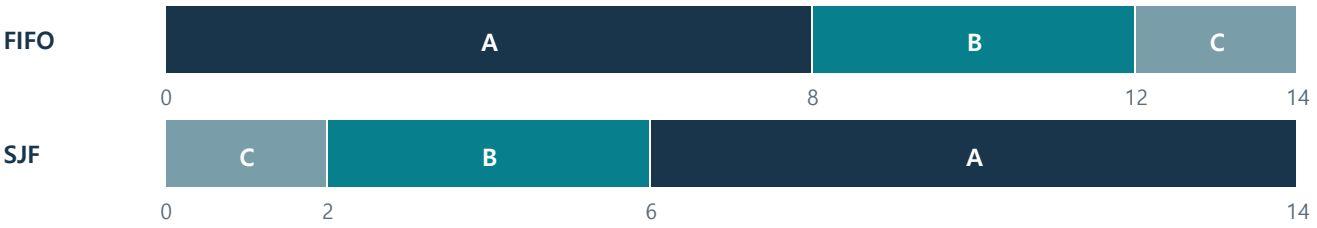
07장 | 어떤 성능 지표를 줄이려는지부터 정한다

> **반환 시간 = 완료 시각 - 도착 시각**
> **응답 시간 = 첫 실행 시각 - 도착 시각**

정책	선택 기준	장점·한계
FIFO / FCFS	먼저 도착한 작업	단순. 긴 작업 뒤 짧은 작업이 기다리는 convoy effect
SJF	가장 짧은 작업; 비선점	동시 도착·실행시간을 아는 조건에서 평균 반환 시간 최적
STCF / SRTF	남은 실행시간 최소; 선점	새 짧은 작업에 즉시 대응. 남은 시간 정보가 필요
RR	준비 큐를 퀀텀 단위로 순환	초기 응답·공정성에 유리. 문맥 교환 비용과 반환 시간 절충

짧은 계산 예제

A=8, B=4, C=2가 시각 0에 도착하고 교환 비용은 0이라고 하자. FIFO의 도착 순서는 A·B·C로 정한다.



FIFO 평균 반환 시간 = (8+12+14)/3 = **11.33**.
SJF 평균 반환 시간 = (2+6+14)/3 = **7.33**.

퀀텀과 I/O의 영향

- 퀀텀이 짧으면 응답은 빨라지지만 교환 횟수가 늘어난다. 길면 교환 비용은 줄지만 RR이 FIFO에 가까워진다.
- I/O로 대기한 작업 대신 다른 준비 작업을 실행하면 CPU 계산과 I/O를 겹쳐 이용률을 높일 수 있다.

SJF·STCF의 최적성은 교환 비용·I/O·실행시간 정보에 관한 가정에 의존한다. 실제 시스템에서는 작업 길이를 모르므로 관측 기반 정책이 필요하다.

숙제

`scheduler.py` 프로그램은 응답 시간, 반환 시간, 총 대기 시간의 관점에서 여러 스케줄러가 어떻게 동작하는지를 볼 수 있게 한다. 자세한 내용은 README를 참조하시오.

문제

1. 길이가 200인 세 개의 작업을 SJF와 FIFO 스케줄링 방식으로 실행할 경우 응답 시간과 반환 시간을 계산하시오.
2. 같은 조건이지만 작업의 길이가 각각 100, 200 및 300일 경우에 대해 계산하시오.
3. 2번과 같은 조건으로 타임 슬라이스가 1인 RR 스케줄러에 대해서도 계산하시오.
4. SJF와 FIFO가 같은 반환 시간을 보이는 워크로드의 유형은 무엇인가?
5. SJF가 RR과 같은 응답 시간을 보이기 위한 워크로드와 타임 퀀텀의 길이는 무엇인가?
6. 작업의 길이가 증가하면 SJF의 응답 시간은 어떻게 되는가? 변화의 추이를 보이기 위해서 시뮬레이터를 사용할 수 있는가?
7. 타임 퀀텀의 길이가 증가하면 RR의 응답 시간은 어떻게 되는가? N 개의 작업이 주어졌을 때, 최악의 응답 시간을 계산하는 식을 만들 수 있는가?

07장 / 정답 · 해설

CPU 스케줄링 기본

시뮬레이션 수치는 교재 저자의 공식 프로그램과 명시한 조건으로 검산했다. 미지정 옵션은 해당 버전의 기본값을 사용한다.

문제 1 · 길이 200인 작업 3개: FIFO와 SJF

답 둘 다 응답 시간 0·200·400, 반환 시간 200·400·600이다. 평균은 각각 **200**, **400**이다.

해설 모두 시각 0에 도착하고 작업 길이가 같으면 어떤 동률 순서로 실행해도 평균이 같다. 교환 비용 0, 비선점 실행을 가정한다.

문제 2 · 길이 100·200·300의 FIFO와 SJF

답 FIFO의 입력 순서가 100→200→300이면 SJF와 같다. 응답 0·100·300, 반환 100·300·600, 평균 응답 **133.33**, 평균 반환 **333.33**이다.

해설 SJF는 오름차순으로 실행한다. FIFO 순서가 300→200→100이면 응답 0·300·500, 반환 300·500·600으로 평균 응답 266.67, 평균 반환 466.67이 된다. 입력 순서를 밝히지 않으면 FIFO 수치가 하나로 정해지지 않는다.

문제 3 · RR, 퀀텀 1

답 A·B·C 순서로 첫 실행은 0·1·2, 완료는 298·499·600이다. 평균 응답 **1**, 평균 반환 **465.67**이다.

해설 첫 99라운드가 297틱을 사용한다. 다음 A 실행으로 298에 A가 끝난다. B·C만 남아 번갈아 실행하다 B가 499에 끝나고 C가 600에 끝난다. 명령 실행 구간과 완료 시각을 혼동하지 않는다.

작업	길이	응답	반환
A	100	0	298
B	200	1	499
C	300	2	600

문제 4 · FIFO와 SJF의 반환 시간이 같은 워크로드

답 동시 도착·비선점 조건에서 FIFO 순서가 길이의 오름차순이면 같다. 길이가 모두 같거나 작업 하나뿐이어도 같다.

해설 두 알고리즘이 동일한 실행 순서를 만들기 때문이다. 특정 평균만 우연히 같은 경우와 모든 작업의 일정이 같은 조건을 구분한다.

문제 5 · SJF와 RR의 응답 시간이 같은 조건

답 동시 도착·같은 정렬 순서에서 RR의 각 첫 퀀텀이 해당 작업을 끝낼 만큼 크면 SJF와 같다. 충분조건은 q≥최대 작업 길이와 SJF 순서의 준비 큐다.

해설 마지막 작업의 길이는 뒤 작업의 초기 응답에 영향이 없으므로, 더 약한 충분조건은 마지막을 제외한 모든 작업이 첫 퀀텀에 끝나는 것이다. 작업 하나만 있는 경우도 항상 같다. q가 충분히 커도 큐 순서가 다

르면 SJF와 같지 않을 수 있다.

문제 6 · 작업 길이가 늘면 SJF 응답 시간은?

답 정렬 순서가 유지되고 앞선 작업들의 길이가 늘면 뒤 작업의 응답 시간이 늘어난다. 모든 길이를 k배 하면 응답·반환 시간도 k배다.

해설 정렬된 길이 L1…Ln에서 i번째 응답은 앞의 i-1개 길이의 합이다. 마지막 작업만 길어지면 그 작업의 최초 응답과 다른 작업의 최초 응답은 변하지 않는다. scheduler.py -l 100,200,300 -p SJF -c와 -l 200,400,600을 비교하면 된다.

문제 7 · RR 퀀텀과 최악의 초기 응답

답 동시에 준비된 N개 작업, 교환 비용 0이면 마지막 작업의 초기 응답은 최대 **(N-1)q**다.

해설 앞의 N-1개 작업이 각각 q씩 사용하면 그만큼 기다린다. 어떤 작업이 일찍 끝나면 더 짧다. 전환당 비용 c를 앞 작업 사이와 목표 작업 진입까지 세면 (N-1)(q+c)로 모델링할 수 있다. 새 도착·다른 우선순위·초기 디스패치 비용이 있으면 그 조건을 추가한다.

08장 / 핵심 요약

멀티 레벨 피드백 큐

08장 | 실행시간을 몰라도 관측한 행동으로 우선순위를 조정

> **MLFQ의 목표**
> 짧은 작업과 대화형 작업에 빠르게 응답하면서, 긴 CPU 작업도 계속 진행하게 한다. 과거 CPU 사용으로 앞으로의 행동을 추정한다.

최종 규칙 다섯 가지

규칙	동작
1	우선순위가 높은 큐의 작업을 먼저 실행한다.
2	같은 우선순위의 작업끼리는 RR로 실행한다.
3	새 작업은 가장 높은 우선순위 큐에서 시작한다.
4	현재 단계의 누적 CPU 할당량을 소진하면 아래 큐로 강등한다. I/O 양보 횟수와 무관하다.
5	주기 S마다 모든 작업을 최상위 큐로 올린다(boost).

왜 규칙 4와 5가 필요한가

문제	보완	결과
기아: 높은 큐가 계속 바쁨	주기적인 우선순위 상승	낮은 큐의 작업도 CPU를 얻음
작업 성격이 달라짐	boost 후 다시 관측	CPU 중심에서 대화형으로 바뀐 작업을 재평가
퀀텀 직전에 I/O로 양보해 강등 회피	단계별 누적 CPU 시간 측정	짧게 쪼개 사용해도 할당량 소진 시 강등

높은 큐는 짧은 퀀텀으로 응답성을, 낮은 큐는 긴 퀀텀으로 교환 비용 감소를 노릴 수 있다. 큐 수·할당량·boost 주기는 워크로드에 맞춰 조정한다.

퀀텀은 한 번에 실행하는 길이이고 **단계별 할당량**은 강등 전 사용할 수 있는 누적 CPU 시간이다. 문제에서 둘의 값을 별도로 주면 구분해서 계산한다.

숙제

프로그램 `mlfq.py`는 지금의 이 장에서 논의한 MLFQ 스케줄러가 어떻게 동작하는지를 살펴볼 수 있게 한다. 자세한 사항은 README를 참조하시오.

문제

1. 두 개의 작업과 두 개의 큐를 무작위로 구성하여 실행시켜 보시오. 각 문제에 대한 MLFQ 실행 추적을 계산하시오. 문제를 쉽게 하기 위해 각 작업의 길이를 제한하고 입출력은 하지 않는다고 가정하시오.
2. 이 장의 예제를 재현하려면 스케줄러를 어떻게 실행해야 하는가?
3. 라운드 로빈 스케줄러처럼 동작시키려면 스케줄러의 매개변수를 어떻게 설정해야 하는가?
4. 두 개의 작업과 스케줄러 매개변수를 가진 워크로드를 고안하시오. 두 작업 중 하나의 작업은 옛날 규칙 4a 및 4b를 이용하여(`-S` 플래그를 켠다) 스케줄러를 자신에게 유리하게 동작하도록 만들어 특정 구간에서 99%의 CPU를 차지하도록 고안해야 한다.
5. 가장 높은 우선순위 큐의 타임 퀀텀의 길이가 10 ms인 시스템이 있다고 하자. 하나의 장기 실행(및 잠재적인 기아 위험) 작업이 적어도 5%의 CPU를 사용할 수 있도록 보장하려면 얼마나 자주 가장 높은 우선순위로 이동시켜야 하는가(`-B` 플래그 켠다)?
6. 스케줄링에서 제기되는 질문 중 하나는, 입출력이 방금 종료된 작업은 큐의 어느 쪽에 추가해야 하는가이다. 플래그 `-I`가 시뮬레이터의 이 행동 양식을 변경한다. 몇 개의 워크로드를 가지고 실험하여 이 플래그의 영향을 확인할 수 있는지 보라.

08장 / 정답 · 해설

멀티 레벨 피드백 큐

시뮬레이션 수치는 교재 저자의 공식 프로그램과 명시한 조건으로 검산했다. 미지정 옵션은 해당 버전의 기본값을 사용한다.

문제 1 · 2개 큐·2개 작업의 무작위 추적

답 예시 조건 -n 2 -j 2 -m 20 -M 0 -q 5, boost 없음, 단계별 할당량 1이다. 검산 추적은 아래와 같다.

해설 Q1이 높은 큐다. 처음 각각 높은 큐에서 최대 5틱씩 실행한 뒤 남으면 Q0으로 강등한다. 표의 연속 구간은 같은 작업의 인접 실행을 합친 것이며, 낮은 큐의 퀀텀 경계가 없어졌다는 뜻은 아니다.

시드	J0,J1 길이	실행 구간 [시작,끝)
0	17,8	0-5: J0(Q1) / 5-10: J1(Q1) / 10-15: J0(Q0) / 15-18: J1(Q0) / 18-25: J0(Q0)
1	3,15	0-3: J0(Q1) / 3-8: J1(Q1) / 8-18: J1(Q0)
2	19,2	0-5: J0(Q1) / 5-7: J1(Q1) / 7-21: J0(Q0)

문제 2 · 본문 예제 재현

답 아래 명령의 -I은 시작 시각, CPU 길이, I/O 빈도를 지정한다. 본문 그림의 특성별 재현 예를 제시했다.

해설 -Q의 값은 높은 큐부터 적는다. 옛 I/O 우선순위 유지 규칙은 -S로, 새 누적 회계는 -S 없이 실행한다. 입출력 시간·boost 주기·큐별 퀀텀을 그림과 일치시켜야 동일한 일정이 된다.

예제	mlfq.py 뒤의 인자
CPU 작업 하나	-I 0,200,0 -Q 10,10,10 -c
나중에 도착한 짧은 작업	-I 0,200,0:100,20,0 -Q 10,10,10 -c
짧은 CPU 버스트와 I/O	-I 0,200,0:100,20,1 -Q 10,10,10 -S -c
주기적 boost	-I 0,200,0:100,50,1:100,50,1 -Q 10,10,10 -S -B 50 -c
새 시간 회계	-I 0,200,0:100,50,9 -Q 10,10,10 -i 1 -c
큐별 퀀텀 차이	-I 0,200,0:0,200,0 -Q 10,20,40 -c

문제 3 · MLFQ를 RR처럼 만들기

답 큐를 하나만 두고 모든 작업에 같은 퀀텀을 적용한다: -n 1 -q 2.

해설 모든 작업이 같은 우선순위이므로 강등이 실행 대상의 우선순위를 달리 만들지 않는다. -I 0,10,0:0,10,0의 응답은 0·2, 반환은 18·20으로 보통 RR과 같다. I/O 완료 앞삽입(-I)은 끄면 기본 FIFO 준비 큐에 더 가깝다.

문제 4 · 옛 규칙을 이용한 CPU 99% 점유

답 예: -l 0,1000,0:0,1000,99 -n 2 -q 100 -i 1 -S -I -c.

해설 CPU 작업 J0은 처음 100틱을 쓰고 낮은 큐로 내려간다. J1은 99틱마다 1틱의 I/O를 하며, -S가 사용 시간을 초기화해 높은 큐에 남는다. 워밍업 뒤 J1의 반복 구간에서 100틱당 약 99틱을 점유한다. 시작·종료를 포함한 전체 생애 몫을 99%라고 주장하는 것은 아니다. -S를 끄면 누적 사용량이 100틱을 넘을 때 강등되어 조작을 막는다.

문제 5 · 10 ms 퀀텀으로 최소 5% 보장

답 매 boost 주기마다 해당 작업이 높은 큐의 10 ms를 온전히 보장받는다는 가정이면 **B≤200 ms**다.

해설 10/B≥0.05이므로 B≤200이다. 다만 작업 수·큐 순서·도착 패턴이 명시되지 않으면 -B 200만으로 무조건 5%를 보장할 수 없다. 매번 10 ms를 받는다는 조건 자체가 필요하다. 예컨대 동시에 21개 작업 모두에게 최소 5%를 보장하는 것은 한 CPU에서 불가능하다.

문제 6 · I/O 완료 작업의 앞삽입·뒤삽입

답 -I를 켜면 I/O가 끝난 작업을 현재 큐의 앞에 넣는다. 예시에서 J1의 반환 시간은 **40→22**로 줄고 J0은 50으로 같다.

해설 검산 조건은 -l 0,40,0:0,10,2 -n 1 -q 5 -i 1이다. 앞삽입은 다시 짧은 CPU 버스트를 실행할 기회를 빨리 줘 I/O를 겹치기 좋다. 같은 우선순위의 CPU 작업은 더 오래 기다릴 수 있어 공정성과 응답성 사이의 절충이 있다.

09장 / 핵심 요약

비례 배분과 CFS

09장 + 보충 | 처리시간보다 CPU 몫의 공정한 배분에 초점

정책	핵심 규칙	특징
Lottery	실행 가능한 작업의 티켓 중 하나를 무작위 추첨	장기 평균은 티켓 비율에 접근. 짧은 구간의 몫은 흔들림
Stride	가장 작은 pass를 선택하고 실행 후 stride를 더함	결정적으로 비율에 접근. 새 작업의 pass 초기화 주의
CFS	가중치를 반영한 vruntime이 작은 작업을 선택	공정성과 교환 비용을 균형 있게 조절

Lottery의 선택 확률 = 내 티켓 / 전체 실행 가능 티켓
Stride: stride = 큰 상수 K / 티켓 수
실행 후 pass = pass + stride

티켓은 사용자별 화폐로 변환하거나, 서버에 일을 맡길 때 전달할 수 있다. Stride에서 티켓 100·50·250과 K=10,000이면 보폭은 100·200·40으로, 작은 보폭의 작업이 더 자주 선택된다.

첨부 판본의 CFS 핵심

- nice가 작을수록 가중치가 크다. 가중치가 큰 작업은 같은 실제 실행시간에도 vruntime이 더 천천히 증가해 더 많은 CPU를 얻는다.

time_slice(i) = sched_latency × weight(i) / Σweight
Δvruntime(i) = 실제 실행시간 × 1024 / weight(i)

- 최소 실행 길이(min_granularity)로 과도한 교환을 막는다. 위 타임 슬라이스 식은 기본 배분 모델이다.
- vruntime 순서의 레드-블랙 트리로 삽입·삭제를 O(log n)에 처리한다. 잠든 작업의 복귀 시 vruntime을 조정해 CPU 독점을 막는다.

nice=-5의 가중치 3121과 nice=0의 1024를 비교하면 CPU 몫은 약 3:1이다. CFS 설명과 매개변수는 원본 보충의 모델을 따른다.

숙제

lottery.py 프로그램은 추첨 스케줄러의 동작을 살펴볼 수 있게 한다. 자세한 사항은 README를 참조하시오.

문제

1. 3개의 작업 및 1, 2, 3의 랜덤 시드를 위한 시뮬레이션을 위한 해결책을 계산하시오.
2. 다음 두 가지 특정 작업을 실행시켜 보라. 각 작업의 길이는 10, 작업 0은 1장의 추첨권, 작업 1은 100장의 추첨권을 가진다(예, -l 10:1,10:100). 추첨권의 개수가 심하게 불균형을 이룰 경우 어떻게 동작하나? 작업 0은 작업 1이 종료하기 전에 실행될 수 있는가? 얼마나 자주 실행될 것인가? 일반적으로 이러한 추첨권의 불균형은 추첨 스케줄링의 동작에 어떤 영향을 미치는가?
3. 작업의 길이가 100이고 100장의 같은 추첨권을 가진 두 작업(-l 100:100,100:100)을 실행시킨다면 스케줄러는 얼마나 불공정한가? (확률적인) 답을 결정하기 위해 몇몇 다른 랜덤 시드를 가지고 실행해 보시오. 불공정성은 하나의 작업이 다른 작업보다 얼마나 일찍 종료하느냐를 기준으로 결정된다고 하자.
4. 타임 퀀텀의 크기(-q)가 커질수록 이전 질문에 대한 대답은 어떻게 바뀔까?
5. 이 장에서 나온 것 같은 그래프를 만들 수 있는가? 탐구해 볼만한 다른 것은 무엇인가? 보폭 스케줄러의 그래프는 어떻게 될까?

09장 / 정답 · 해설

비례 배분과 CFS

시뮬레이션 수치는 교재 저자의 공식 프로그램과 명시한 조건으로 검산했다. 미지정 옵션은 해당 버전의 기본값을 사용한다.

문제 1 · 시드 1·2·3의 추첨 일정

답 기본값: 작업 3개, 최대 길이 10, 최대 티켓 100, 퀀텀 1이다. 전체 실행 순서와 완료 시각은 아래와 같다.

해설 티켓의 누적 구간에 당첨 번호가 들어간 작업이 실행된다. 끝난 작업의 티켓은 제거하므로 다음 추첨의 전체 티켓 수가 줄어든다. 숫자는 작업 번호이며, 1개 숫자당 1틱 실행이다.

시드	작업: 길이/티켓	실행 순서	완료 시각
1	J0: 1/84 / J1: 7/25 / J2: 4/44	2 0 1 2 2 2 1 1 1 1 1 1	J0=2, J2=6, J1=12
2	J0: 9/94 / J1: 8/73 / J2: 6/30	2 0 0 2 0 1 0 2 0 0 0 1 0 0 1 2 1 1 1 2 1 1 2	J0=14, J1=22, J2=23
3	J0: 2/54 / J1: 3/60 / J2: 6/6	1 1 0 1 0 2 2 2 2 2 2	J1=4, J0=5, J2=11

문제 2 · 티켓 1 대 100의 불균형

답 두 작업이 함께 준비되어 있을 때 J0의 선택 확률은 1/101 0.99%다. J1이 끝나기 전에 J0이 실행될 수도 있다.

해설 J1이 10번 연속 뽑혀 먼저 끝나는 확률은 (100/101)^10이다. 그 전에 J0이 적어도 한 번 실행될 확률은 약 9.47%다. 시드 1·3에서는 J1이 10틱에, 시드 2에서는 11틱에 끝나므로 시드 2에서 J0도 미리 한 번 실행됐다. 최종 J0 완료는 모두 20틱이다. 낮은 확률은 실행 금지가 아니며 짧은 구간의 공정성은 보장되지 않는다.

문제 3 · 같은 길이·같은 티켓의 불공정성

답 완료 시간 비율 U=빠른 완료/느린 완료로 정의하면, U가 1에 가까울수록 공정하다. 검산 수치는 아래와 같다.

해설 길이가 각각 100이므로 총 완료 시간은 200틱이다. 동일 티켓이어도 추첨 결과가 흔들려 한 작업이 먼저 끝난다. 몇 개의 시드 결과를 전체 확률 분포로 단정하지 말고 여러 시드의 평균·분산을 확인한다.

시드	J0 완료	J1 완료	U
1	200	196	0.980
2	200	190	0.950
3	196	200	0.980

문제 4 · 퀀텀이 커질 때

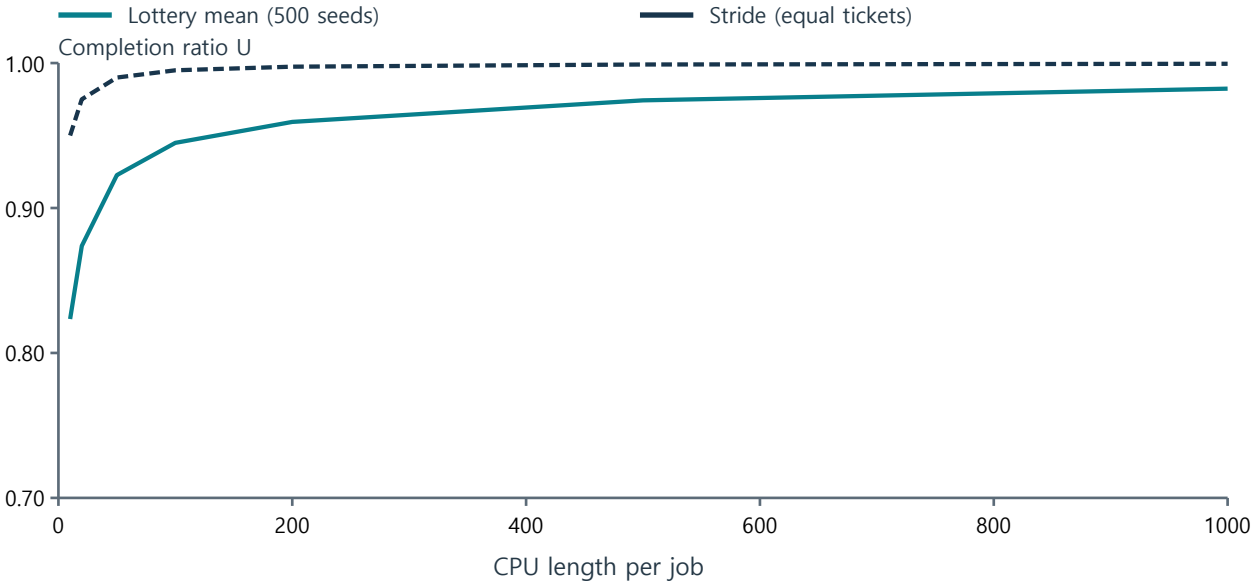
답 보통 단기 불공정성이 커진다. q=10의 시드 1·2·3에서 U는 각각 0.800·0.950·0.950이다.

해설 같은 총 CPU 시간에서 추첨 횟수가 줄어 평균화 효과가 약해지고, 한 번 당첨된 작업이 큰 CPU 몫을 받는다. 다만 모든 개별 시드에서 반드시 더 나빠지는 것은 아니다. q가 작업 길이보다 커지면 먼저 당첨된 작업이 통째로 끝나 RR과 같은 교대 효과도 사라진다.

문제 5 · 그래프와 Stride 비교

답 아래는 동일 티켓 작업 2개, q=1에서 길이별 500개 시드의 평균 U다. Stride는 결정적 교대로 더 빠르게 1에 가까워진다.

해설 추첨 모형은 공식 코드의 난수·티켓 계산을 그대로 구현하고 시드 1·2·3의 완료 시간으로 대조 검증했다. 동일 길이 L의 Stride 교대는 완료가 2L-1, 2L이므로 U=1-1/(2L)다. 다른 티켓 비율, 퀀텀, 새 작업 도착 시 pass 초기화도 추가 실험 대상으로 적절하다.



실행 길이와 완료 시간 공정성. Lottery는 시드 0-499 평균, Stride는 이론값.

10장 / 핵심 요약

멀티프로세서 스케줄링

10장 | 부하 균형·캐시 친화성·동기화를 함께 고려

CPU가 여러 개면 실행 대상을 고르는 것뿐 아니라 "어느 CPU에 배치할까"가 중요하다. 같은 프로그램이 한 CPU에 다시 실행되면 캐시·TLB에 남은 정보를 재사용할 수 있다.

먼저 구분할 세 가지

개념	역할
캐시 일관성(coherence)	여러 캐시의 같은 메모리 데이터가 모순되지 않도록 하드웨어가 관리
캐시 친화성(affinity)	같은 CPU에서 재실행해 캐시·TLB 재사용률을 높이는 성능 전략
동기화	공유 큐 등 자료 구조의 복합 연산을 락 등으로 보호

단일 큐와 CPU별 큐

구성	장점	단점·보완
SQMS: 공유 큐 하나	구현이 단순하고 유휴 CPU가 작업을 가져가기 쉬움	공유 락 경쟁, 확장성 저하, 친화성 관리 필요
MQMS: CPU별 큐	락 경쟁 감소, 높은 친화성, 확장에 유리	한 CPU만 바쁘고 다른 CPU가 놀 수 있음. 이주로 부하 균형

> **Work stealing**
> 한 CPU가 다른 CPU의 큐를 살펴 작업을 가져온다. 너무 자주 살피면 동기화 비용이 커지고, 너무 드물게 살피면 부하 불균형이 오래 남는다.

판단 순서

- 준비 작업이 충분한데 유휴 CPU가 있으면 큐 분배와 이주 정책을 확인한다.
- CPU를 옮겼을 때 처리속도가 떨어지면 캐시·TLB 재가열 비용과 작업 집합 크기를 확인한다.
- CPU 수를 늘려도 성능이 늘지 않으면 공유 락 경쟁과 메모리 병목을 확인한다.

캐시 일관성이 유지되어도 읽기-수정-쓰기 같은 여러 단계의 공유 자료 갱신에는 별도의 동기화가 필요하다.

멀티프로세서 스케줄링 - 영어판 추가 내용 번역

멀티프로세서 스케줄링(고급)

영어판에만 있는 추가 내용 번역

대조 기준: 한국어판 전체와 Version 1.10 영어 원문을 문단 단위로 비교했습니다. 이미 한국어판에 있는 내용은 반복하지 않고, 영어판에만 있는 내용만 번역했습니다. 절 번호는 영어 원문의 번호를 유지했습니다.

영어판에만 있는 참고문헌

[TTG95] Josep Torrellas, Andrew Tucker, Anoop Gupta, 'Evaluating the Performance of Cache-Affinity Scheduling in Shared-Memory Multiprocessors', Journal of Parallel and Distributed Computing, Vol. 24, No. 2, 1995. 캐시 친화성 스케줄링을 처음 다룬 논문은 아니지만, 더 이른 연구에 대한 인용을 포함하며 앞선 큐잉 기반 분석 논문보다 읽기 쉽고 실용적이다.

숙제: 시뮬레이션

이 실습에서는 `multi.py`를 사용하여 멀티프로세서 CPU 스케줄러를 시뮬레이션하고 세부 동작을 학습한다. 시뮬레이터와 선택 사항에 관한 자세한 내용은 관련 README를 참고하라.

1. 먼저 시뮬레이터로 효과적인 멀티프로세서 스케줄러를 구성하는 방법을 알아보자. 첫 시뮬레이션에서는 실행 시간이 30이고 작업 집합 크기가 200인 작업 하나만 실행한다. 다음과 같이 하나의 가상 CPU에서 작업 a를 실행하라: `./multi.py -n 1 -L a:30:200`. 완료하는 데 얼마나 걸리는가? `-c` 플래그로 최종 답을 확인하고, `-t` 플래그로 매 틱마다 작업이 어떻게 스케줄되는지 추적하라.

2. 이제 기본 크기가 100인 캐시를 키워서 크기 200인 작업 집합이 캐시에 들어가게 하라. 예: `./multi.py -n 1 -L a:30:200 -M 300`. 작업이 캐시에 들어간 뒤 얼마나 빠르게 실행될지 예측할 수 있는가? 힌트: `-r` 플래그로 정하는 warm rate 매개변수를 기억하라. `-c` 플래그를 켜서 답을 확인하라.

3. `multi.py`의 장점 중 하나는 여러 추적 플래그로 내부 동작을 더 자세히 볼 수 있다는 점이다. 위와 같은 시뮬레이션을 실행하되 이번에는 남은 실행 시간 추적 옵션 `-T`를 켜라. 이 플래그는 각 시간 단계에서 CPU에 스케줄된 작업과, 한 틱 실행 후 그 작업에 남은 실행 시간을 함께 보여 준다. 두 번째 열이 감소하는 방식에서 무엇을 발견할 수 있는가?

4. 각 작업에 대한 CPU 캐시의 상태를 표시하도록 `-C` 플래그도 추가하라. 각 캐시는 해당 작업에 대해 차가운 상태면 빈칸, 따뜻한 상태면 w를 표시한다. 이 간단한 예에서 작업 a의 캐시는 언제 따뜻해지는가? 준비 시간 매개변수 `-w`를 기본값보다 작거나 크게 바꾸면 어떻게 되는가?

추가 번역 1쪽

멀티프로세서 스케줄링 - 영어판 추가 내용 번역

5 이제 단일 CPU에서 단일 작업을 실행하는 시뮬레이터의 동작을 이해했을 것이다. 멀티프로세서 CPU 스케줄링을 살펴보자. 두 CPU 시스템에서 다음 세 작업을 실행하라: `./multi.py -n 2 -L a:100:100,b:100:50,c:100:50`. 중앙 집중식 라운드 로빈 스케줄러라면 완료하는 데 얼마나 걸릴지 예측하라. `-c`로 답을 확인하고, `-t`로 단계별 동작을 살펴본 뒤 `-C`로 각 작업의 캐시가 효율적으로 따뜻해졌는지 확인하라. 무엇을 발견했는가?

6 이제 이 장에서 설명한 캐시 친화성을 연구하기 위해 명시적인 제어를 적용한다. `-A` 플래그는 스케줄러가 특정 작업을 배치할 수 있는 CPU를 제한한다. 작업 b와 c는 CPU 1에, a는 CPU 0에만 배치해 보자: `./multi.py -n 2 -L a:100:100,b:100:50,c:100:50 -A a:0,b:1,c:1`. 여러 추적 옵션도 함께 사용하라. 이 버전은 얼마나 빨리 실행될지 예측할 수 있는가? 왜 더 나은 성능을 내는가? a, b, c를 두 프로세서에 다르게 배치하면 더 빨라지거나 느려지는가?

7 캐시를 가진 멀티프로세서에서는 여러 CPU와 캐시를 사용할 때 작업이 예상보다 더 크게 가속될 수 있다. N개의 CPU에서 실행했을 때 속도가 N배를 넘는 경우를 초선형 가속(super-linear speedup)이라고 한다. 세 작업 `-L a:100:100,b:100:100,c:100:100`을 작은 캐시 `-M 50`에서 생성하고, CPU가 1개, 2개, 3개인 시스템(`-n 1`, `-n 2`, `-n 3`)에서 실행하라. 이후 CPU별 캐시 크기를 100으로 키워 같은 실험을 반복하라. CPU 수가 늘어날 때 성능에서 무엇을 발견할 수 있는가? `-c`로 예측을 확인하고 다른 추적 플래그로 더 자세히 분석하라.

8 시뮬레이터에서 살펴볼 또 다른 기능은 CPU별 스케줄링 옵션인 `-p` 플래그다. 다시 두 CPU와 세 작업 구성 `-L a:100:100,b:100:50,c:100:50`을 사용하라. 이 방식은 앞에서 직접 설정한 친화성 제한과 비교해 어떻게 동작하는가? 엿보기 간격(peek interval)인 `-P` 값을 더 작게 또는 크게 바꾸면 성능이 어떻게 달라지는가? CPU 수가 늘어날 때 CPU별 스케줄링 방식은 어떻게 동작하는가?

9 마지막으로 무작위 워크로드를 생성하고, 서로 다른 프로세서 수, 캐시 크기, 스케줄링 옵션에서 성능을 예측해 보라. 이 과정을 반복하면 멀티프로세서 스케줄링을 충분히 익힐 수 있을 것이다.

추가 번역 2쪽

10장 / 정답 · 해설

멀티프로세서 스케줄링

시뮬레이션 수치는 교재 저자의 공식 프로그램과 명시한 조건으로 검산했다. 미지정 옵션은 해당 버전의 기본값을 사용한다.

문제 1 · 캐시보다 큰 작업 집합

답 a:30:200, CPU 1개, 캐시 크기 100이면 **30틱**이다.

해설 작업 집합 200이 캐시 100에 들어가지 않아 warm 상태가 되지 않는다. 1틱에 남은 실행량이 1씩 줄어든다.

문제 2 · 캐시를 300으로 늘리기

답 기본 warmup=10, warm rate=2에서 **20틱**이다.

해설 첫 10틱은 속도 1로 10만큼 처리하고, 나머지 실행량 20은 속도 2로 10틱에 처리한다. 10+(30-10)/2=20이다. 캐시를 키워도 워밍업 이전부터 빨라지는 것은 아니다.

문제 3 · 남은 시간 -T의 변화

답 차가울 때는 1씩, 따뜻해진 뒤에는 2씩 줄어든다.

해설 이 시뮬레이터에서 실행 길이는 실시간 틱 수가 아니라 처리해야 할 일의 양이다. -r=2이면 warm CPU가 한 틱에 처리하는 일의 양이 2가 된다. 마지막 양이 2보다 작으면 마지막 틱을 모두 사용한 것으로 표시할 수 있다.

문제 4 · warmup 시간 -w의 영향

답 캐시 300에서 기본 w=10이면 20틱, w=5이면 18틱, w=20이면 25틱이다.

해설 warmup 실행량을 채운 뒤 캐시 표시가 w가 되고 빠른 실행이 시작된다. 완료 시간은 이 예에서 w+ceil((30-w)/2)로 계산한다. 워밍업이 작업 길이보다 길거나 캐시에 안 들어가면 가속 효과를 얻지 못한다.

문제 5 · 공유 RR 큐의 2 CPU 일정

답 a:100:100,b:100:50,c:100:50, 기본 캐시 100에서 **150틱**이다.

해설 큐를 돌면서 작업이 CPU 사이를 이동하고, 한 CPU에 새 작업이 들어오면 기존 캐시 상태를 밀어낼 수 있다. 이 일정은 각 CPU의 warm 비율 0%다. 총 일 300을 속도 1인 CPU 둘이 처리해 150틱이 걸린다.

문제 6 · 친화성 지정의 효과

답 -A a:0,b:1,c:1이면 **110틱**이다. 반대로 a·b를 CPU0, c를 CPU1에 고정하면 **200틱**이다.

해설 좋은 배치에서는 CPU0의 a는 55틱에 끝나고, CPU1의 b·c 작업 집합 합 100이 캐시에 들어가 110틱에 끝난다. 나쁜 배치의 a·b 합 150은 캐시 100을 초과해 그 CPU가 200틱을 쓴다. 큐에 작업 수가 비슷해도 작업 집합 크기와 캐시 배치에 따라 결과가 달라진다.

문제 7 · 초선형 가속

답 캐시 50: CPU 1·2·3개에서 300·150·100틱. 캐시 100: 300·150·55틱이다.

해설 각 작업의 집합은 100이다. 캐시 100과 CPU 3개면 작업마다 CPU를 하나씩 써서 warm 상태를 유지한다. 단일 CPU 300틱과 비교한 가속은 300/55 5.45배로 CPU 3개보다 크다. CPU 수뿐 아니라 사용 가능한 캐시와 재사용 패턴도 함께 바뀐 결과다.

캐시/CPU	1개	2개	3개
50	300	150	100
100	300	150	55

문제 8 · CPU별 큐와 peek 간격

답 동일 작업과 기본 시드에서 -p의 P=0·1·5·30·100 결과는 200·90·90·100·130틱이다.

해설 P=0은 훔치기를 끄므로 초기 배치의 불균형을 고치지 못한다. 더 자주 살피면 부하를 빠르게 맞추지만 실제 구현에서는 락·탐색·이주 비용이 든다. 이 시뮬레이터는 그 비용 전체를 모델링하지 않으므로 P=1이 실제 시스템에서도 언제나 최선이라고 결론 내리지 않는다.

P	총 틱
0	200
1	90
5	90
30	100
100	130

문제 9 · 무작위 워크로드 비교

답 예시 -j 3 -R 40 -W 100 -M 100, 공유 큐와 기본 나머지 설정의 완료 시간은 아래와 같다.

해설 작업 길이, 캐시 적합성, 이동 횟수를 함께 본다. 작업이 3개뿐인 이 예에서 CPU 4개는 추가 유휴 CPU와 다른 이동 패턴 때문에 2개보다 오히려 느릴 수 있다. CPU를 늘리면 언제나 단조롭게 빨라진다는 가정은 성립하지 않는다.

시드	CPU1	CPU2	CPU4
1	31	20	25
2	41	22	30
3	26	16	20

13장 / 핵심 요약

주소 공간

> **주소 공간**은 프로세스가 보는 가상 메모리다. 물리 배치와 사용 방식을 분리해 투명성·효율성·보호와 격리를 제공한다.

영역	주요 내용
코드·정적 데이터	명령어, 전역·정적 변수
스택	함수 호출 문맥, 지역 변수, 반환 주소
힙	동적 할당 데이터
매핑 영역	공유 라이브러리, 파일·익명 메모리 등

가상 주소와 물리 주소

각 프로세스의 같은 가상 주소는 서로 다른 물리 프레임을 가리킬 수 있다. 운영체제와 MMU가 주소를 변환하고 권한을 검사한다. 프로세스가 자신의 가상 주소만 알아도 프로그램을 작성할 수 있다.

단순 그림과 실제 배치

힙과 스택이 서로 반대 방향으로 자라는 그림은 설명용 관례다. 실제 프로세스는 로더·여러 라이브러리·익명 매핑 등을 포함하며 ASLR 때문에 배치도 달라질 수 있다.

관측할 때의 구분

지표	의미
가상 크기	주소 공간에 매핑된 범위
RSS	현재 RAM에 상주한 페이지의 용량
free	현재 미사용 RAM
available	회수 가능한 캐시 등을 고려한 추가 사용 가능 추정치

malloc으로 큰 가상 범위를 확보했어도 실제 페이지를 쓰기 전에는 RSS가 작을 수 있다. free는 시스템 전체, pmap은 선택한 프로세스의 매핑을 보는 도구다.

원본 PDF 112-122쪽. 다음에 해당 장의 문제 원문과 번호별 답·해설을 수록했다.

Homework (Code) - 한국어 번역

이번 숙제에서는 Linux 기반 시스템에서 가상 메모리 사용량을 확인할 수 있는 몇 가지 유용한 도구를 간단히 배운다. 여기서는 가능한 기능의 일부만 살펴보며, 제대로 익히려면 직접 더 깊게 공부해야 한다.

문제

1. 가장 먼저 살펴볼 Linux 도구는 매우 간단한 free이다. 먼저 man free를 입력하고 매뉴얼 페이지 전체를 읽어보자. 내용이 짧으니 걱정하지 않아도 된다.

2. 이제 free를 실행해보자. 필요하다면 -m과 같은 옵션을 사용해도 된다. -m은 메모리 용량을 MB 단위로 표시한다. 시스템에는 전체 메모리가 얼마나 있는가? 사용 가능한 메모리는 얼마나 되는가? 이 수치들이 예상했던 것과 일치하는가?

3. 다음으로 일정량의 메모리를 사용하는 간단한 프로그램 memory-user.c를 작성하라. 이 프로그램은 명령행 인자 하나를 입력받아야 하며, 이 값은 프로그램이 사용할 메모리의 크기(MB)를 의미한다. 프로그램을 실행하면 지정된 크기의 배열을 할당하고, 배열 전체를 계속 순회하면서 각 원소에 접근해야 한다. 이 작업을 무한히 반복하도록 만들거나, 명령행에서 실행 시간을 추가로 입력받아 일정 시간 동안만 실행하도록 만들어도 된다.

4. memory-user 프로그램을 실행한 상태에서 같은 컴퓨터의 다른 터미널 창을 열어 free를 실행하라. 프로그램 실행 중 메모리 사용량이 어떻게 변하는가? memory-user 프로그램을 종료하면 어떻게 변하는가? 결과가 예상과 일치하는가? 서로 다른 크기의 메모리를 사용하도록 설정하여 실험해보라. 아주 큰 양의 메모리를 사용하면 어떤 일이 발생하는가?

5. 이번에는 pmap이라는 도구를 사용해보자. 시간을 들여 pmap의 매뉴얼 페이지를 자세히 읽어보라.

6. pmap을 사용하려면 조사하려는 프로세스의 프로세스 ID(PID)를 알아야 한다. 먼저 ps auxw를 실행하여 모든 프로세스의 목록을 확인한다. 그런 다음 웹 브라우저처럼 관심 있는 프로세스 하나를 선택한다. 앞에서 작성한 memory-user 프로그램을 사용해도 된다. 편의를 위해 memory-user 프로그램에서 getpid()를 호출하여 자신의 PID를 출력하도록 만들어도 된다.

7. 이제 선택한 프로세스에 대해 pmap을 실행해보라. -X 같은 여러 옵션을 사용하면 프로세스에 대한 더 자세한 정보를 확인할 수 있다. 어떤 정보들이 나타나는가? 우리가 단순하게 배운 코드(code) / 스택(stack) / 힙(heap) 구조와 비교했을 때, 실제 현대 프로세스의 주소 공간은 얼마나 다양한 영역들로 구성되어 있는가?

8. 마지막으로 서로 다른 크기의 메모리를 사용하도록 memory-user를 실행하고, 각각에 대해 pmap을 실행해보라. 어떤 차이가 나타나는가? pmap의 출력 결과가 예상했던 것과 일치하는가?

13장 / 정답 · 해설

주소 공간

코드는 Linux/POSIX용 예시다. 이 Windows 작업 환경에서 Linux 하드웨어 수치를 실측하지 않았다. 실측 문항은 방법·해석·예상 경향을 답으로 제시한다.

문제 1 · free 매뉴얼과 필드

답 total은 전체 사용 가능한 메모리, free는 현재 미사용 메모리, available은 스왑 없이 새 작업에 쓸 수 있을 것으로 추정한 메모리다.

해설 free와 available은 다르다. available은 회수 가능한 캐시 등을 고려한다. used·buff/cache·shared·swap도 함께 확인한다. 도구 버전에 따라 used 계산식과 출력이 달라질 수 있어 실제 man free를 기준으로 해석한다. free -m은 MiB 단위, free -h는 읽기 쉬운 단위다.

문제 2 · 내 시스템의 전체·사용 가능 메모리

답 숫자는 해당 Linux 시스템에서 free -m 또는 free -h로 측정해야 한다. 이 문서에서는 사용자 시스템의 값을 임의로 넣지 않았다.

해설 설치된 RAM과 total이 정확히 같지 않을 수 있다. 커널·장치 예약과 시스템의 관측 범위를 고려하고, "프로그램이 추가로 쓸 수 있는 양"에는 available을 본다. /proc/meminfo의 MemTotal·MemAvailable과 대조하면 된다.

문제 3 · memory-user.c 작성

답 다음은 입력한 MiB를 할당하고 전체 바이트를 계속 접근하는 프로그램이다. 두 번째 인자로 실행 시간을 제한할 수 있다.

해설 volatile 접근으로 반복문 제거를 막고, 처음 memset으로 실제 페이지에 접근한다. PID를 출력해 pmap에 사용할 수 있다. 크기 계산 오버플로와 malloc 실패를 확인한다. 무한 반복 대신 기본 10초를 사용했다.

gcc -O2 memory-user.c -o memory-user / ./memory-user 64 10

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <time.h>
#include <unistd.h>
static double now(void) {
    struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t);
    return t.tv_sec+t.tv_nsec/1e9;
}
int main(int argc,char **argv) {
    if(argc<2) return 1;
    unsigned long long mb=strtoull(argv[1],NULL,10);
    if(mb==0 || mb>SIZE_MAX/(1024u*1024u)) return 1;
    size_t n=(size_t)mb*1024u*1024u;
    double seconds=argc>2 ? atof(argv[2]) : 10;
    if(seconds<=0) return 1;
    unsigned char *p=malloc(n);
    if(!p) { perror("malloc"); return 1; }
```

```
    memset(p,0,n);
    volatile unsigned char *a=p;
    printf("pid=%ld bytes=%zu\n",(long)getpid(),n); fflush(stdout);
    double end=now()+seconds;
    do {
        for(size_t i=0;i<n;i++) a[i]=(unsigned char)(a[i]+1);
    } while(now()<end);
    free(p); return 0;
}
```

문제 4 · 실행·종료와 큰 메모리 요청

답 페이지를 실제로 접근하면 메모리 사용이 늘고 available이 감소한다. 종료하면 그 프로세스의 자원이 회수된다.

해설 malloc 성공만으로 요청한 모든 물리 페이지가 즉시 상주하는 것은 아니다. 매우 큰 요청은 malloc 실패, 스왑 증가·쓰래싱, 메모리 부족으로 인한 종료 등으로 이어질 수 있다. 캐시 회수·다른 프로세스·커널 메타데이터 때문에 free의 변화가 요청량과 바이트 단위로 딱 맞을 필요는 없다.

문제 5 · pmap이 보여 주는 것

답 한 프로세스의 가상 메모리 매핑을 보여 준다. 주소·영역 크기·권한·매핑 파일 등을 확인한다.

해설 pmap PID의 기본 출력과 pmap -x PID의 상세 출력을 비교한다. -X의 추가 열은 커널과 procps 버전에 따라 다를 수 있다. 가상 매핑 크기와 RAM에 상주한 RSS를 구분한다.

문제 6 · PID 찾기

답 ps auxw에서 PID 열을 확인하거나 프로그램이 출력한 getpid() 값을 사용한다.

해설 이름이 같아도 다른 프로세스가 여러 개 있을 수 있다. 정확한 PID를 지정하고 프로그램이 끝나기 전에 pmap을 실행한다. 예: ./memory-user 64 30을 실행하고 다른 터미널에서 pmap -x <출력된 PID>를 실행한다.

문제 7 · 실제 주소 공간이 복잡한 이유

답 실행 파일의 코드·데이터, 힙, 스택, 공유 라이브러리, 익명 매핑, 파일 매핑, 로더, 커널 지원 영역 등 여러 매핑이 보인다.

해설 단순한 코드/힙/스택 그림은 개념 모델이다. 권한이 r-x인 코드와 rw인 데이터처럼 같은 파일도 여러 영역으로 나뉠 수 있다. RSS 합은 공유 페이지를 중복해서 세는 등 특성이 있으므로 다른 프로세스와 무조건 합산하지 않는다.

문제 8 · 크기를 바꾼 pmap 결과

답 더 큰 할당을 하고 전부 접근하면 관련 익명·힙 매핑의 크기와 RSS가 대체로 증가한다.

해설 작은 할당은 기존 힙 안에서 충족되고 큰 할당은 별도 mmap 영역으로 나타날 수 있다. 할당기의 문턱·재사용·페이지 반올림 때문에 한 매핑이 항상 요청량만큼 정확히 늘어나는 것은 아니다. 아직 접근하지 않은 매핑의 RSS는 가상 크기보다 작을 수 있다.

14장 / 핵심 요약

메모리 API

> **malloc/free는 라이브러리 함수**다. 할당기는 기존 힙을 재사용하며 필요하면 brk/sbrk·mmap 등으로 운영체제에 공간을 요청한다.

API	핵심
malloc(n)	n바이트 요청; 실패 시 NULL; 내용은 자동 0 초기화되지 않음
calloc	할당 후 0으로 초기화
realloc	크기 변경; 주소 이동 가능; 실패 시 기존 블록은 유지
free(p)	할당 블록의 시작 포인터 또는 NULL을 전달

스택·힙과 수명

지역 포인터 변수는 스택에 있어도 가리키는 malloc 블록은 힙에 있다. 스택의 지역 변수는 함수 복귀 후 유효하지 않으며, 힙 블록은 필요가 끝나면 명시적으로 해제한다.

반드시 피할 오류

오류	원인
크기 오류	sizeof(포인터)와 대상 크기 혼동, 문자열 널 바이트 누락
미초기화 읽기	할당했다고 내용까지 유효해지는 것은 아님
범위 밖 접근	100개 배열의 인덱스 100 등
수명 오류	메모리 누수, use-after-free, double free
잘못된 free	블록 중간·스택 포인터 등을 전달

realloc 결과는 임시 포인터로 받고 성공 후 원래 포인터를 바꾼다. 용량을 두 배씩 늘리는 벡터는 분할 상환 삽입 비용을 줄인다. gdb는 실행 상태, Memcheck는 접근 오류·누수 이력 파악에 도움이 된다.

원본 PDF 123-133쪽. 다음에 해당 장의 문제 원문과 번호별 답·해설을 수록했다.

숙제 (코드)

이 과제에서 메모리 할당에 대한 어느 정도의 지식을 얻을 수 있다. 먼저 몇몇 버그 있는 프로그램을 작성한다(재미있겠지!). 다음, 당신이 삽입한 버그를 발견하는 데 도움을 주는 몇 가지 도구를 사용한다. 그러면 이러한 도구가 얼마나 대단한지 알게 될 것이고 미래에 그들을 사용하게 되어 행복하고 생산적인 프로그래머가 될 것이다.

사용할 첫 번째 도구는 `gdb` 디버거이다. 이 디버거에 대해 배울 것이 많다. 여기서는 단지 수박 겉핥기식으로 넘어갈 것이다.

두 번째 사용할 도구는 `valgrind` [SN05]이다. 이 도구는 프로그램의 메모리 누수와 다른 교묘한 메모리 문제를 발견하는 데 도움을 준다. 만약 설치되어 있지 않다면 다음 web 사이트에 가서 설치하라.

`http://valgrind.org/downloads/current.html`

문제

1. `null.c`라는 간단한 프로그램을 작성하라. 이 프로그램은 정수를 가리키는 포인터를 만들고 NULL로 초기화하고 그것을 역참조하려고 한다. `null`이라는 실행 파일이 생성되도록 컴파일하라. 프로그램을 실행시키면 어떤 일이 발생하는가? 프로그램 죽음
2. 다음에 이 프로그램을 심볼 정보가 포함되도록 컴파일하라(`-g` 플래그 사용). 실행 파일에 더 많은 정보를 추가하여 디버거가 변수 이름 등의 유용한 정보를 접근할 수 있게 한다. 디버거에서 프로그램을 실행하라(`gdb null`). `gdb`가 실행되면, `run` 명령어를 입력하라. `gdb`는 무엇을 보여주는가?
3. 마지막으로, 이 프로그램에 `valgrind` 도구를 사용하라. 우리는 `valgrind`의 일부인 `memcheck` 도구를 사용하여 어떤 일이 일어나는지 분석할 것이다. 다음과 같은 명령어를 실행시켜라: `valgrind --leak-check=yes null`. 이를 실행하면 어떻게 되는가? 출력을 해석할 수 있는가?
4. `malloc()`을 사용하여 메모리를 할당하지만 종료 전에 해제하는 것을 잊어버리는 간단한 프로그램을 작성하라. 이 프로그램을 실행시키면 무슨 일이 일어나는가? 이 프로그램의 문제를 gdb를 사용하여 발견할 수 있는가? `valgrind`는 어떤가(다시 `--leak-check=yes` 플래그를 주고 실행)?
5. 크기가 100인 `data`라는 이름의 정수 배열을 만드는 프로그램을 작성하라. 그런 후 `data[I00]`을 0으로 클리어 하라. 이 프로그램을 실행하면 어떤 일이 벌어지는가? `valgrind`를 사용하여 프로그램을 실행하면 어떤 일이 일어나는가? 프로그램은 정확한가?
6. 정수 배열을 할당하고(위와 동일), 다시 해제하고, 배열의 원소 하나의 값을 출력하려고 시도하는 프로그램을 작성하라. 프로그램이 실행되는가? `Valgrind`를 사용하여 실행하면 어떤 일이 발생하는가?

7. `free`에 위에서 할당한 배열의 중간을 가리키는 포인터와 같은 황당한 값을 전달하라. 어떤 일이 벌어지는가? 이런 종류의 문제를 발견하기 위해 도구가 필요한가?

8. 메모리를 할당할 때 다른 인터페이스를 사용해 보라. 예를 들어, 간단한 벡터-비슷한 자료 구조를 만들고 벡터를 관리하기 위하여 `realloc()`을 사용하는 루틴을 만들어라. 벡터 원소를 저장하기 위하여 배열을 사용하라. 사용자가 벡터에 항목을 추가할 때, `realloc()`을 사용하여 필요 공간을 확보하라. 이 벡터 프로그램은 얼마나 잘 동작하는가? 연결 리스트와 비교하면 어떠한가? 버그를 발견하기 위하여 valgrind를 사용하라.

9. `gdb`와 `valgrind` 사용법을 배우기 위하여 시간과 노력을 투자하라. 도구를 잘 아는 것은 매우 중요하다. UNIX와 C 환경에서 디버거 전문가가 되기 위해 시간과 노력을 투자하라.

14장 / 정답 · 해설

메모리 API

코드는 Linux/POSIX용 예시다. 이 Windows 작업 환경에서 Linux 하드웨어 수치를 실측하지 않았다. 실측 문항은 방법·해석·예상 경향을 답으로 제시한다.

문제 1 · NULL 역참조

답 C에서 정의되지 않은 동작이다. 일반적인 Linux에서 0번 페이지가 매핑되지 않아 SIGSEGV로 끝나는 모습을 관찰한다.

해설 "반드시 언제나 같은 오류가 난다"는 C 언어의 보장은 아니다. gcc -O0 -g로 아래 예를 빌드해 관찰한다. 최적화·실행 환경에 따라 결과가 달라질 수 있다.

null.c: 의도적으로 잘못된 코드

```
#include <stddef.h>
int main(void) {
    volatile int *p=NULL;
    return *p; /* intentionally invalid */
}
```

문제 2 · gdb가 보여 주는 정보

답 gcc -O0 -g null.c -o null 후 gdb ./null, run을 실행하면 보통 SIGSEGV와 실패한 소스 행을 볼 수 있다.

해설 bt로 호출 스택, print p로 포인터 값을 본다. -g는 이름·행 정보를 제공할 뿐 버그를 고치지 않는다. 디버거를 사용했다고 프로그램의 메모리 접근이 합법적으로 바뀌는 것도 아니다.

문제 3 · Valgrind Memcheck의 NULL 보고

답 valgrind --leak-check=yes ./null은 보통 주소 0의 잘못된 읽기와 그 크기, 발생 위치를 보고한다.

해설 이 문제의 중심은 누수가 아니라 Invalid read다. 누수 점검 옵션이 있어도 Memcheck는 잘못된 접근을 함께 찾는다. SIGSEGV가 발생하면 종료 원인도 보인다. 구체적인 행 번호·메시지는 빌드와 버전에 따라 달라진다.

문제 4 · malloc한 뒤 free하지 않기

답 프로그램은 정상 종료할 수도 있다. gdb가 자동으로 누수 목록을 보여 주지는 않지만 Memcheck는 누수 상태를 추적한다.

해설 종료 때 OS가 메모리를 회수해도 장시간 실행 중의 누수는 문제다. 포인터가 완전히 사라지면 definitely lost, 종료까지 도달 가능한 포인터가 남으면 still reachable로 분류될 수 있다. 따라서 무조건 definitely lost라고 쓰지 않는다.

gcc -O0 -g leak.c -o leak / valgrind --leak-check=full ./leak

```
#include <stdlib.h>
int main(void) {
    int *p=malloc(100*sizeof *p);
    if(!p) return 1;
    p[0]=1;
    p=NULL; /* intentionally lose the allocation */
    return 0;
}
```

문제 5 · 100개 배열의 data[100]

답 범위 밖 쓰기다. 유효한 인덱스는 0-99이며 프로그램은 올바르지 않다.

해설 바로 충돌하지 않아도 인접 데이터나 할당기 정보를 망가뜨릴 수 있다. malloc으로 얻은 배열에 쓰면 Memcheck는 블록 끝 바로 뒤의 Invalid write를 보고한다. 원문의 data[I00]은 data[100]으로 읽는다.

main 내부의 오류 예시

```
int *data=malloc(100*sizeof *data);
if(!data) return 1;
data[100]=0; /* intentionally out of bounds */
free(data);
```

문제 6 · free 후 배열 접근

답 use-after-free이며 정의되지 않은 동작이다. 이전 값이 우연히 출력돼도 정답 코드가 아니다.

해설 해제된 공간은 재사용되거나 내용이 달라질 수 있다. Memcheck는 해제된 블록에 대한 Invalid read와 할당·해제 위치를 보여 준다.

main 내부의 오류 예시

```
int *data=malloc(100*sizeof *data);
if(!data) return 1;
data[0]=7;
free(data);
printf("%d\n",data[0]); /* intentionally invalid */
```

문제 7 · 배열 중간 포인터를 free

답 free(data+1)은 잘못된 해제다. 할당 함수가 반환한 시작 포인터 또는 NULL을 전달해야 한다.

해설 보통 할당기 오류로 중단되거나 Memcheck가 Invalid free를 보고하지만, UB이므로 모든 환경에서 같은 결과가 보장되지 않는다. 바로 중단되면 도구 없이 증상을 알 수 있어도 도구는 원인을 찾는 데 도움이 된다.

main 내부의 오류 예시

```
int *data=malloc(100*sizeof *data);
if(!data) return 1;
free(data+1); /* intentionally invalid */
```

문제 8 · realloc 기반 벡터

답 매번 1개씩 늘리기보다 용량을 2배로 늘리면 push가 분할 상환 O(1), 인덱스 접근 O(1)이고 연속 배치로 캐시 사용이 좋다.

해설 재할당 한 번은 복사 때문에 O(n)일 수 있다. 실패하면 기존 블록은 살아 있으므로 임시 포인터로 받고 성공 후 바꾼다. 연결 리스트는 연속 공간이 필요 없고 이미 찾은 위치의 연결 변경이 쉽지만 인덱스 탐색은 O(n), 노드 메타데이터와 지역성 비용이 든다.

vector.c: n=100, cap=128, last=99의 논리적 결과

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
typedef struct { int *a; size_t n,cap; } Vec;
static int push(Vec *v,int x) {
    if(v->n==v->cap) {
```

```
        if(v->cap>SIZE_MAX/2) return -1;
        size_t cap=v->cap ? 2*v->cap : 4;
        if(cap>SIZE_MAX/sizeof *v->a) return -1;
        int *tmp=realloc(v->a,cap*sizeof *v->a);
        if(!tmp) return -1;
        v->a=tmp; v->cap=cap;
    }
    v->a[v->n++]=x; return 0;
}
int main(void) {
    Vec v={0};
    for(int i=0;i<100;i++) {
        if(push(&v,i)<0) { free(v.a); return 1; }
    }
    printf("n=%zu cap=%zu last=%d\n",v.n,v.cap,v.a[v.n-1]);
    free(v.a); return 0;
}
```

문제 9 · 디버거와 메모리 도구 익히기

답 gdb는 실패 순간의 실행 상태를, Memcheck는 잘못된 접근·누수의 이력을 찾는 데 적합하다.

해설 재현 입력을 고정하고 -g로 빌드 → gdb run/bt/print → valgrind --leak-check=full --track-origins=yes → 수정 후 같은 입력으로 재확인한다. 정상 실행만으로 메모리 안전성을 증명할 수는 없다. 이 과제는 고정 숫자 답이 없는 실습이다.

15장 / 핵심 요약

주소 변환의 원리

> **동적 재배치**
> 0≤VA<bound이면 PA=base+VA. 여기서 bound는 합법적인 가상 구간의 크기를 뜻한다.

MMU가 모든 메모리 참조를 검사·변환한다. base/bound 변경은 특권 연산이다. base는 물리 시작점, bound는 보호 범위를 정한다.

검사	조건
가상 구간	VA<bound; 끝 주소는 bound-1
물리 배치	base+bound≤물리 메모리 크기
변환	합법적인 VA에 base를 더함

짧은 예제

base=32 KiB, bound=16 KiB, VA=1 KiB이면 PA=33 KiB다. VA=16 KiB는 bound와 같아 위반이다. 주소가 범위 안에 있어도 물리 배치 자체가 RAM 끝을 넘으면 설정이 잘못됐다.

하드웨어와 OS의 역할

하드웨어	운영체제
변환·범위 검사	프로세스 생성·종료 때 물리 공간 관리
사용자/커널 모드	문맥 교환 때 base/bound 저장·복원
특권 레지스터·예외	오류 접근 처리, 중단한 주소 공간 이동 후 base 갱신

장점과 한계

작은 하드웨어로 빠른 변환과 보호를 제공한다. 하지만 주소 공간 전체를 연속 물리 구간에 배치하면 힙-스택 사이 미사용 범위도 낭비할 수 있다. 세그멘테이션은 필요한 논리 영역별 관리로 확장한다.

원본 PDF 134-147쪽. 다음에 해당 장의 문제 원문과 번호별 답·해설을 수록했다.

숙제

프로그램 `relocation.py`는 베이스와 바운드 레지스터를 가진 시스템에서 주소 변환이 어떻게 일어나는지 볼 수 있게 한다. 자세한 내용은 README를 참조하시오.

문제

1. 시드 1, 2 및 3을 가지고 실행하고 프로세스에 의해 생성된 각 가상 주소가 범위 내에 있는지 계산하라. 바운드 안에 있다면 주소 변환을 수행하라.
2. 다음과 같은 플래그를 주고 실행하라: `-s 0 -n 10`. 생성된 모든 가상 주소가 범위 안에 있는 것을 보장하기 위해서는 -l을 어떤 값으로 설정해야 하는가?
3. 다음과 같은 플래그를 주고 실행하라: `-s 1 -n 10 -l 100`. 주소 공간 전체가 여전히 물리 메모리에 들어가려면 설정할 수 있는 바운드 레지스터의 최댓값은 얼마인가?
4. 더 큰 주소 공간(`-a`)과 물리 메모리(`-p`)를 설정하여 위 문제와 동일하게 실행시켜 보아라.
5. 바운드 레지스터의 값이 변함에 따라 임의로 생성된 가상 주소 중 유효한 주소의 비율은 얼마인가? 다른 랜덤 시드를 가지고 실행한 결과를 그래프로 나타내시오. 값의 범위는 0부터 주소 공간의 최대 크기로 한다.

15장 / 정답 · 해설

주소 변환의 원리

시뮬레이션 수치는 교재 저자의 공식 프로그램과 명시한 조건으로 검산했다. 미지정 옵션은 해당 버전의 기본값을 사용한다.

문제 1 · 시드 1·2·3 주소 변환

답 VA<limit이면 PA=base+VA, 그 외에는 위반이다. 기본 A=1024, 물리 메모리 P=16384의 결과는 아래와 같다.

해설 유효 주소의 끝은 limit-1이다. 같은 시드에서도 base·limit을 직접 지정하면 난수 소비 순서가 달라져 이후 VA열이 달라질 수 있으므로 생성된 주소를 반드시 확인한다.

시드	base	limit	VA	PA/결과
1	13884	290	782	위반
1	13884	290	261	14145
1	13884	290	507	위반
1	13884	290	460	위반
1	13884	290	667	위반
2	15529	500	57	15586
2	15529	500	86	15615
2	15529	500	855	위반
2	15529	500	753	위반
2	15529	500	685	위반
3	8916	316	378	위반
3	8916	316	618	위반
3	8916	316	640	위반
3	8916	316	67	8983
3	8916	316	13	8929

문제 2 · 모든 생성 주소를 유효하게 만드는 limit

답 이 조건의 최소 limit은 **930**이다. 가장 큰 생성 주소 929를 포함하려면 1을 더해야 한다.

해설 기본 -s 0 -n 10의 VA는 430,265,523,414,802,310,488,597,929,516이다. -l 930을 명시하면 난수 소비가 달라져 앞에 776이 추가되는 등 주소열이 바뀌지만 여전히 최대는 929다. -l 929이면 929는 위반이다. 모든 가능한 0-1023 주소를 보장하려면 limit=1024를 써야 한다.

문제 3 · base 최댓값: 원문 번역 차이

답 영문 원문은 **base의 최댓값**을 묻는다. limit=100, P=16384이면 base≤P-limit이므로 **16284**다.

해설 base=16284는 물리 주소 16284-16383에 100바이트 보호 구간을 놓는다. 16285는 끝이 넘쳐 프로그램이 오류를 낸다. 첨부 한국어의 "바운드 레지스터 최댓값"은 영문의 base와 다르다. 한국어를 문자 그대로 읽어 기본 base=2201을 고정하면 물리적으로 가능한 limit은 14183이지만, 논리적 주소 공간 A=1024 안에서는 limit=1024면 이미 모든 주소를 포함한다. 1024바이트 전체 슬롯 배치를 별도로 요구하면 base≤15360이다. 이 답은 시뮬레이터가 검사하는 base+limit≤P 조건을 기준으로 한다.

문제 4 · 큰 주소 공간으로 재실행

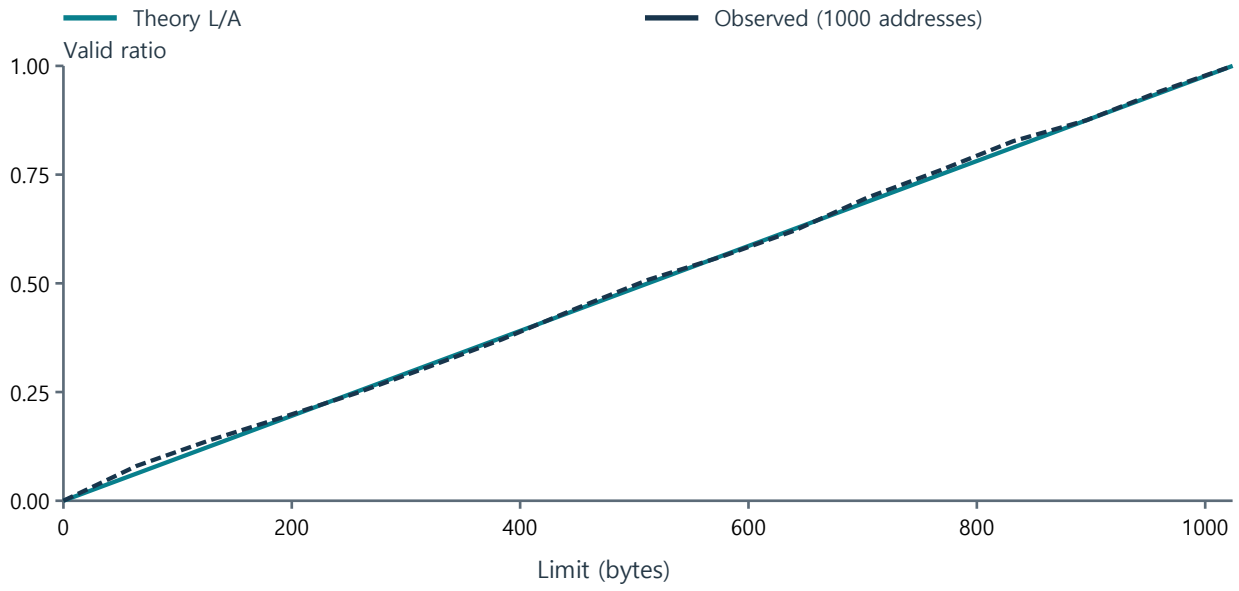
답 예: -s 1 -n 10 -a 4k -p 64k -b 32768 -l 4k -c. 생성 주소 10개가 모두 유효하다.

해설 전체 VA 범위 0-4095가 limit=4096 안에 들어간다. VA=550이면 PA=33318, VA=3471이면 PA=36239다. 물리 공간 조건도 32768+4096≤65536으로 만족한다. 크기를 바꿔도 범위 검사와 더하기 원리는 같다.

문제 5 · limit에 따른 유효 주소 비율

답 VA가 0...A-1에서 균등하면 P(valid)=min(limit,A)/A다. 0≤limit≤A에서는 직선 limit/A다.

해설 아래는 A=1024, base=0, 시드 0의 VA 1000개를 고정해 측정한 표본 비율과 이론값이다. 난수·base·주소열을 고정해야 limit의 효과만 비교할 수 있다. 유한 표본은 직선 근처에서 흔들린다. 원점과 limit=A의 끝점은 각각 0과 1이다.



유효 주소 비율: 이론과 고정된 시드 0의 1000개 주소.

16장 / 핵심 요약

세그멘테이션

> **세그멘테이션**: 코드·힙·스택 등 논리 영역마다 base, 크기, 성장 방향, 보호 권한을 둔다. 사용하지 않는 큰 빈 영역을 RAM에 배치하지 않아도 된다.

구분	범위 검사·변환
순방향	0≤offset<L, PA=base+offset
역방향: base가 끝의 다음 주소	-L≤offset<0, PA=base+offset

가상 주소 상위 비트 등으로 세그먼트를 찾고 나머지 비트에서 영역 내 위치를 얻는다. 역방향 스택은 기준 주소에서 음수 오프셋을 사용한다. 베이스의 정의에 따라 경계식을 확인한다.

공유와 보호

읽기/쓰기/실행 권한을 세그먼트별로 검사한다. 읽기 전용 코드 세그먼트를 여러 주소 공간에 매핑하면 프로세스의 논리적 독립성을 유지하며 물리 메모리를 절약한다.

외부 단편화

가변 크기의 연속 구간을 배치하므로 빈 공간이 흩어진다. 빈 합계 24 KiB가 3개로 분리돼 있으면 20 KiB 연속 요청을 못 받을 수 있다. 압축으로 합칠 수 있지만 데이터 복사와 레지스터 변경 비용이 든다.

방식	강점	대가
base/bound 전체 공간	단순한 변환·보호	미사용 가상 범위의 물리 낭비
세그멘테이션	논리 영역별 배치·공유	외부 단편화, 크기 성장·빈 공간 관리

고정 크기 페이지는 분산된 프레임을 사용할 수 있어 기본 배치의 외부 단편화 문제를 줄인다. 페이지 테이블과 TLB라는 새로운 비용도 필요하다.

원본 PDF 148-159쪽. 다음에 해당 장의 문제 원문과 번호별 답·해설을 수록했다.

숙제

이 프로그램은 세그멘테이션 시스템에서 주소 변환이 어떤 식으로 일어나는지 보여준다. 자세한 내용은 README를 참조하시오.

문제

1. 우선 몇 주소를 변환하기 위해 작은 주소 공간을 사용하여 보자. 여기에 여러 가지 랜덤 시드와 함께 간단한 매개변수의 집합이 있다. 이때 주소를 변환할 수 있을까?

```
segmentation.py -a 128 -p 512 -b 0 -l 20 -B 512 -L 20 -s 0
segmentation.py -a 128 -p 512 -b 0 -l 20 -B 512 -L 20 -s 1
segmentation.py -a 128 -p 512 -b 0 -l 20 -B 512 -L 20 -s 2
```

2. 이제 우리가 구축한 이 작은 주소 공간을 이해하고 있는지 알아보자(위의 질문에서 사용한 매개변수를 사용하여). 세그멘트 0의 합법적인 가상 주소의 최댓값은 무엇인가? 세그멘트 1의 합법적인 가상 주소의 최저값은 무엇인가? 이 전체 주소 공간의 잘못된 주소 중 가장 큰 주소와 가장 작은 주소는 무엇인가? 마지막으로, 플래그 `-A`와 함께 실행한 `segmentation.py`의 정확성을 어떻게 확인할 수 있을까?

3. 16바이트 주소 공간과 128바이트의 물리 메모리가 있다고 하자. 시뮬레이터가 지정된 주소 스트림에 대해 다음과 같은 변환 결과를 생성하기 위해서는 베이스와 바운드를 어떤 값으로 설정해야 하는가: 유효, 유효, 위반, ..., 위반, 유효, 유효? 다음과 같은 매개변수를 가정한다.

```
segmentation.py -a 16 -p 128
        -A 0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15
        --b0 ? --l0 ? --b1 ? --l1 ?
```

4. 무작위로 생성된 가상 주소 중 약 90%가 유효한 시스템에서(즉, 세그멘테이션 위반이 아닌) 문제를 만들고 싶다고 가정하자. 그런 결과를 내려면 시뮬레이터를 어떻게 설정해야 하는가? 어떤 매개변수가 중요한가?

5. 모든 가상 주소가 유효하지 않도록 시뮬레이터를 실행할 수 있는가? 어떻게?

16장 / 정답 · 해설

세그멘테이션

시뮬레이션 수치는 교재 저자의 공식 프로그램과 명시한 조건으로 검산했다. 미지정 옵션은 해당 버전의 기본값을 사용한다.

문제 1 · 128바이트 주소 공간의 두 세그먼트

답 SEG0은 VA 0-19에서 PA=VA, SEG1은 VA 108-127에서 PA=512+(VA-128)이다.

해설 VA 상위 비트로 두 영역을 구분한다. 두 번째 세그먼트의 base=512는 물리 구간 끝의 다음 주소를 가리키므로 음수 오프셋을 더한다. 108의 오프셋 -20도 유효해 PA=492다.

시드	VA	세그먼트	PA/결과
0	108	1	492
0	97	1	위반
0	53	0	위반
0	33	0	위반
0	65	1	위반
1	17	0	17
1	108	1	492
1	97	1	위반
1	32	0	위반
1	63	0	위반
2	122	1	506
2	121	1	505
2	7	0	7
2	10	0	10
2	106	1	위반

문제 2 · 경계 주소와 -A 검사

답 SEG0의 최대 유효 VA=**19**, SEG1의 최소 유효 VA=**108**. 잘못된 VA의 최소·최대는 **20·107**이다.

해설 -A 19,20,107,108,127 -c로 유효·위반·위반·유효·유효를 확인한다. 전체 공간을 보려면 -A에 0...127을 명시한다. 역방향 세그먼트의 조건은 -L≤음수 offset<0이며, 정방향의 0≤offset<l과 경계 표현이 다르다.

문제 3 · 처음·끝 두 주소만 유효하게 만들기

답 예: --b0 0 --l0 2 --b1 128 --l1 2.

해설 VA 0·1은 PA 0·1로, VA 14·15는 PA 126·127로 변환되고 VA 2-13은 위반이다. 베이스 위치는 두 물리 구간이 합법적이고 겹치지 않도록 다른 값을 선택해도 된다. 유효 여부 패턴을 결정하는 핵심은 두 limit=2 다.

문제 4 · 약 90%가 유효한 설정

답 겹치지 않는 두 유효 가상 구간의 크기 합을 주소 공간의 약 90%로 잡는다. 예: A=1000, l0=l1=450.

해설 -a 1000 -p 4096 --b0 0 --l0 450 --b1 4096 --l1 450이면 이론상 900/1000=90%다. 기본 시드 0의 1000개 주소 검산은 903개 유효, 97개 위반이다. 유한 표본이 정확히 90%일 필요는 없다. limit은 각 절반 크기 이하여야 하고 물리 구간도 겹치지 않아야 한다.

문제 5 · 모든 주소를 무효로 만들기

답 두 limit을 0으로 설정하면 된다: --l0 0 --l1 0.

해설 -a 16 -p 128 --b0 0 --b1 128과 함께 VA 0-15를 모두 지정하면 전부 위반이다. 유효한 길이가 0이므로 어떤 주소도 들어갈 수 없다. 베이스만 엉뚱한 값으로 놓아 설정 자체를 실패시키는 것과 구분한다.

17장 / 핵심 요약

빈 공간 관리

17장 | 요청을 빠르게 처리하면서 단편화를 줄이는 방법

단편화	의미	예
내부 단편화	할당한 블록 안에서 사용하지 못한 공간	7 KiB 요청에 8 KiB 블록 할당
외부 단편화	빈 공간이 흩어져 큰 연속 요청을 충족하지 못함	10+10 KiB가 떨어져 있어 15 KiB 요청 실패

기본 메커니즘

- **분할(splitting)**: 큰 빈 블록에서 요청량을 떼고 나머지를 빈 공간으로 유지한다.
- **병합(coalescing)**: 해제된 블록과 주소상 인접한 빈 블록을 합친다.
- **헤더**: 크기 등 관리 정보를 저장해 free가 포인터만으로 해제 크기를 알 수 있게 한다. 요청 크기 외에 헤더·정렬 비용도 고려한다.

할당 정책 비교

정책	선택 규칙	주의점
Best fit	요청을 수용하는 가장 작은 블록	검색 비용과 작은 잔여 조각 발생
Worst fit	가장 큰 블록	큰 조각을 남기려 하지만 항상 좋은 결과는 아님
First fit	순서대로 보아 처음 맞는 블록	검색이 빠를 수 있으나 앞부분에 단편 집중
Next fit	직전 탐색 위치에서 이어 탐색	탐색을 분산. 리스트 순서에 영향받음

빈 블록 순서가 10·30·20이고 요청이 15라면 Best fit은 20, Worst fit과 First fit은 30을 고른다. 헤더·정렬 비용을 생략한 예다.

개선 방법

크기별 리스트는 자주 쓰는 크기를 빠르게 할당한다. **슬랩**은 특정 객체를 재사용해 초기화 비용을 줄인다. **버디**는 2의 거듭제곱 크기로 분할하고 짝이 비었을 때 재귀 병합한다. 버디는 병합이 쉽지만 내부 단편화가 생긴다.

일반 C 힙 할당기는 이미 사용자에게 준 포인터를 자유롭게 옮길 수 없으므로, 물리 세그먼트 압축과 같은 방식으로 해결하기 어렵다.

숙제

`malloc.py` 프로그램은 이 장에서 논의한 간단한 빈 공간 할당기의 동작을 탐색할 수 있게 한다. 그 기본 동작에 대한 자세한 사항은 README를 참조하시오

문제

1. 먼저 무작위 할당과 해제를 생성하기 위하여 `-n 10 -H 0 -p BEST -s 0` 플래그를 주고 실행시켜라. `alloc()`과 `free()`가 무엇을 반환할지 예측할 수 있는가? 각 요청 후의 빈 공간 리스트의 상태를 추측할 수 있는가? 시간이 지남에 따라 빈 공간 리스트에 대해 무엇을 알 수 있는가?
2. 빈 공간 리스트를 탐색하기 위하여 최악 적합 정책을 사용할 때 결과는 어떻게 달라지는가(`-p WORST`)? 무엇이 바뀌는가?
3. 최초 적합을 사용한 경우는 어떤가(`-p FIRST`)? 최초 적합을 사용하면 속도 향상은 어떻게 되는가?
4. 위 질문에 대해, 몇몇 정책의 경우 리스트의 정렬 순서가 빈 위치를 찾는 데 걸리는 시간에 영향을 준다. 다른 빈 공간 리스트의 순서(`-l ADDRSORT`, `-l SIZESORT+`, `-l SIZESORT-`)를 사용하여 정책과 리스트 순서가 서로 어떤 영향을 주는지 관찰하라.
5. 빈 공간의 합병은 매우 중요할 수 있다. 무작위 할당의 개수를 늘려라(`-n 1000`). 시간이 지날수록 요청의 개수가 많아지면 어떤 현상이 발생하는가? 병합이 있는 채로 그리고 없는 채로 실행하라(`-C` 플래그가 없거나 있거나). 출력에 어떤 차이를 볼 수 있는가? 각각의 경우의 시간이 지남에 따라 빈 공간 리스트의 크기는 얼마나 커지는가? 이 경우 리스트의 정렬 순서가 중요한가?
6. 할당 비율 `-P`를 50 이상으로 변경하면 어떤 일이 일어날까? 100에 가까울수록 할당에는 어떤 일이 생기는가? 0에 가까워진다면?
7. 심하게 단편화된 빈 공간을 만들어 내려면 어떤 종류의 요구를 생성할 수 있는가? 단편화된 리스트를 생성하기 위하여 `-A` 플래그를 사용하라. 다른 정책과 옵션이 빈 공간 리스트의 구조에 어떤 영향을 주는지 관찰하라.

17장 / 정답 · 해설

빈 공간 관리

시뮬레이션 수치는 교재 저자의 공식 프로그램과 명시한 조건으로 검산했다. 미지정 옵션은 해당 버전의 기본값을 사용한다.

문제 1 · BEST의 10개 연산 추적

답 기본 힙 100바이트, base=1000, 헤더 0, 병합 없음의 결과는 아래와 같다. 빈 목록은 (주소,크기)로 표시했다.

해설 할당은 해당 크기를 떼어 시작 주소를 반환하고, 정상 free는 0을 반환한다. 사용량이 줄어도 병합하지 않아 이웃한 빈 블록이 별개의 원소로 남는다. 외부 단편화가 커지면 빈 공간 합계가 충분해도 큰 요청을 만족시키기 어렵다.

연산	반환	연산 후 빈 목록
ptr[0] = Alloc(3)	1000	(1003,97)
Free(ptr[0])	0	(1000,3) (1003,97)
ptr[1] = Alloc(5)	1003	(1000,3) (1008,92)
Free(ptr[1])	0	(1000,3) (1003,5) (1008,92)
ptr[2] = Alloc(8)	1008	(1000,3) (1003,5) (1016,84)
Free(ptr[2])	0	(1000,3) (1003,5) (1008,8) (1016,84)
ptr[3] = Alloc(8)	1008	(1000,3) (1003,5) (1016,84)
Free(ptr[3])	0	(1000,3) (1003,5) (1008,8) (1016,84)
ptr[4] = Alloc(2)	1000	(1002,1) (1003,5) (1008,8) (1016,84)
ptr[5] = Alloc(7)	1008	(1002,1) (1003,5) (1015,1) (1016,84)

문제 2 · WORST로 바꾸기

답 처음 세 번의 할당·해제는 같지만, 네 번째 Alloc(8)은 1008 대신 가장 큰 블록의 1016을 고른다.

해설 뒤의 Alloc(2)·Alloc(7)은 1024·1026을 반환한다. 최종 빈 목록은 (1000,3),(1003,5),(1008,8),(1016,8),(1033,67)이다. 6번의 할당에서 총 20개 원소를 검사한다. 가장 큰 구간을 쓰므로 항상 단편화가 줄어드는 것은 아니다.

문제 3 · FIRST와 탐색 비용

답 ADDRSORT의 FIRST는 이 예에서 BEST와 같은 주소를 반환하되 검사 수가 1·2·3·3·1·3, 합 **13**으로 줄어든다. BEST는 18이다.

해설 FIRST는 맞는 원소를 만나면 중단한다. BEST는 더 작은 적합 후보를 찾으려고 끝까지 본다. 일반적으로 두 정책의 반환 주소가 항상 같지는 않으며, 이 짧은 예제에서만 같은 결과다.

문제 4 · 정렬 순서와 정책

답 이 구현에서 BEST·WORST는 정렬되어 있어도 전체를 검색한다. FIRST는 리스트 순서의 영향을 직접 받는다.

해설 FIRST+SIZESORT-는 이 예의 6개 할당에서 모두 첫 원소가 맞아 총 6번 검사한다. 주소순·크기 오름차순은 총 13번이다. 큰 블록을 먼저 쓰는 동작은 WORST와 비슷해지지만 탐색 방식은 다르다. 같은 크기의 동률 선택과 반환 주소도 정렬에 따라 달라질 수 있다.

정책	ADDRSORT	SIZESORT+	SIZESORT-
BEST	18	18	18
WORST	20	20	20
FIRST	13	13	6

문제 5 · 1000개 연산과 병합

답 BEST·시드 0·헤더 0·할당 비율 50%에서 병합과 정렬별 결과는 아래와 같다.

해설 주소순 병합은 단편 수와 실패 횟수를 크게 줄인다. 이 시뮬레이터의 병합은 리스트에서 연속한 원소를 검사하므로 크기순 목록에서는 물리적으로 인접한 청크가 떨어져 있어 잘 합쳐지지 않는다. 이는 이 구현의 한계이며, 모든 실제 할당기가 병합을 위해 주소순 단일 리스트를 써야 한다는 뜻은 아니다.

순서	병합	최종 빈 블록 수	할당 실패
ADDRSORT	없음	31	177
ADDRSORT	있음	1	29
SIZESORT+	없음	31	177
SIZESORT+	있음	28	163
SIZESORT-	없음	31	177
SIZESORT-	있음	33	170

문제 6 · 할당 비율 P의 영향

답 할당 비율이 높으면 사용 중인 블록이 늘어 공간 부족 실패가 많아진다. 낮으면 해제가 우세해 빈 공간이 유지된다.

해설 같은 1000개 연산, BEST·시드 0·ADDRSORT·병합에서 P=10·50·75·100의 실패는 0·29·456·983이다. P=100에서는 한 번도 해제하지 않아 대부분 요청이 실패한다. 이 프로그램은 P=0을 assertion으로 거부하므로 0에 가까운 효과는 P=1·10 등으로 관찰한다. "초기부터 해제만"은 해제할 블록이 없다는 점도 고려한다.

문제 7 · 심한 외부 단편화 만들기

답 100바이트를 10바이트 블록 10개로 채운 뒤 짝수 번호 블록만 해제하고 15바이트를 요청한다.

해설 빈 합계 50바이트지만 최대 연속 빈 구간은 10이므로 마지막 요청은 -1이다. 사용 중인 블록이 사이에 있어 병합을 켜도 15바이트 구간이 생기지 않는다. 모두 같은 크기라 BEST·WORST·FIRST의 마지막 실패는 같다.

시뮬레이터 명령: 필요하면 한 줄로 입력

```
malloc.py -S 100 -H 0   -A +10,+10,+10,+10,+10,+10,+10,+10,+10,+10,-0,-2,-4,-6,-8,+15 -c
```

18장 / 핵심 요약

페이징과 페이지 테이블

18장 | 고정 크기 단위로 나누고 물리 배치를 분리

가상 페이지(page)와 **물리 프레임(frame)**을 같은 고정 크기로 나눈다. 가상 페이지는 RAM의 임의의 빈 프레임에 배치할 수 있다. 프로세스별 페이지 테이블은 VPN을 PFN으로 연결한다.

> **VA = VPN × 페이지 크기 + offset**
> **PA = PFN × 페이지 크기 + offset**
> **offset은 주소 변환 전후 동일하다.**

주소 변환 계산

32비트 VA, 페이지 4 KiB=2^{12}이면 offset 12비트, VPN 20비트다. VA=0x1234는 VPN=1, offset=0x234. VPN 1이 PFN 7에 매핑되면 PA=**0x7234**다.

PTE에 들어가는 정보

정보	의미
PFN	RAM에 있는 물리 프레임 번호
valid	프로세스 주소 공간에서 유효한 페이지인지(교재의 추상 모델)
protection	읽기·쓰기·실행 및 사용자/커널 접근 권한
present	현재 물리 메모리에 있는지
dirty / accessed	내용이 수정됐는지 / 접근된 적이 있는지

페이징이 해결한 문제와 남은 문제

- 같은 크기의 프레임에 분산 배치하므로 기본 페이지 배치에서 외부 단편화를 없앤다. 마지막 페이지 등의 내부 낭비는 남는다.
- 선형 페이지 테이블은 커질 수 있다. 32비트·4 KiB·PTE 4바이트면 2^{20} × 4 = **4 MiB/프로세스**다.
- TLB가 없으면 단순 데이터 접근도 PTE 읽기와 실제 데이터 읽기를 거쳐 추가 비용이 생긴다. 다음 두 장이 속도와 공간 문제를 해결한다.

valid·present 등은 의미를 구분하기 위한 교재 모델이다. 실제 CPU의 PTE 비트 구성과 비트 이름이 항상 일대일로 대응하는 것은 아니다.

숙제

이 과제에서는 `paging-linear-translate.py`라는 간단한 프로그램을 사용하게 될 것이다. 이 프로그램을 이용하여 선형 페이지 테이블을 이용한 단순한 가상-대-물리 주소 변환이 어떻게 작동하는지를 이해하고 있는지확인할 수 있다. 자세한 사항은 README를 참조하시오.

문제

1. 변환을 하기 전에 매개변수가 주어졌을 때 선형 페이지 테이블이 크기를 변경하는 방법을 연구하기 위해 시뮬레이터를 사용해 보자. 매개변수가 바뀜에 따라 선형 페이지 테이블의 크기를 계산하시오. 추천하는 입력은 아래와 같다. `-v` 플래그를 사용하여 얼마나 많은 페이지 테이블 항목이 채워지는지 알 수 있다.

   먼저, 주소 공간이 커짐에 따라 선형 페이지 테이블 크기의 변화를 이해하기 위하여:

   ```
   paging-linear-translate.py -P 1K -a 1m -p 512m -v -n 0
   paging-linear-translate.py -P 1K -a 2m -p 512m -v -n 0
   paging-linear-translate.py -P 1K -a 4m -p 512m -v -n 0
   ```

   그런 다음 페이지 크기가 커짐에 따라 선형 페이지 테이블 크기의 변화를 이해하기 위하여:

   ```
   paging-linear-translate.py -P 1k -a 1m -p 512m -v -n 0
   paging-linear-translate.py -P 2k -a 1m -p 512m -v -n 0
   paging-linear-translate.py -P 4k -a 1m -p 512m -v -n 0
   ```

   어느 것 하나라도 실행하기 전에 예상 경향에 대해 생각해 보라. 주소 공간이 커지면 페이지 테이블 크기는 어떻게 변하는가? 페이지 크기가 커지는 경우는? 일반적으로 큰 페이지 크기를 사용하면 안 되는 이유는 무엇인가?

2. 자 이제 변환을 해 보자. 몇 가지 작은 예부터 시작해서 `-u` 플래그를 사용하여 주소 공간에 할당된 페이지 개수를 변경해 보라. 예를 들면:

   ```
   paging-linear-translate.py -P 1k -a 16k -p 32K -v -u 0
   paging-linear-translate.py -P 1k -a 16k -p 32K -v -u 25
   paging-linear-translate.py -P 1k -a 16k -p 32K -v -u 50
   paging-linear-translate.py -P 1k -a 16k -p 32K -v -u 75
   paging-linear-translate.py -P 1k -a 16k -p 32K -v -u 100
   ```

   각 주소 공간에 할당된 페이지의 비율을 높인다면 어떤 일이 벌어지는가?

3. 그러면 몇 가지 무작위 배정을 시도하자. 몇 가지 다른 (그리고 때로는 정말 어이 없는) 주소 공간 매개변수를 사용해 보라:

   ```
   paging-linear-translate.py -P 8  -a 32   -p 1024 -v -s 1
   paging-linear-translate.py -P 8K -a 32k  -p 1m   -v -s 2
   paging-linear-translate.py -P 1m -a 256m -p 512m -v -s 3
   ```

이 조합 중 어느 조합이 비현실적인가? 이유는?

4. 다른 문제를 시험해 보기 위해 프로그램을 사용하라. 프로그램이 더 이상 작동하지 않는 제약을 찾을 수 있는가? 예를 들어, 만약 주소 공간의 크기가 물리 메모리보다 *크다면* 어떤 일이 벌어지는가?

18장 / 정답 · 해설

페이징

시뮬레이션 수치는 교재 저자의 공식 프로그램과 명시한 조건으로 검산했다. 미지정 옵션은 해당 버전의 기본값을 사용한다.

문제 1 · 주소 공간·페이지 크기와 선형 테이블

답 PTE를 4바이트로 가정하면 테이블 크기는 (A/P)×4다. 주소 공간은 비례, 페이지 크기는 반비례로 영향을 준다.

해설 큰 페이지는 PTE 수와 TLB 부담을 줄이지만 내부 단편화, 큰 할당 단위, 필요한 일부만 접근할 때의 I/O·메모리 낭비가 커질 수 있다. 물리 메모리 512 MiB 자체가 아래 가상 페이지 수를 바꾸지는 않는다.

A	P	항목 수	테이블 크기
1 MiB	1 KiB	1024	4 KiB
2 MiB	1 KiB	2048	8 KiB
4 MiB	1 KiB	4096	16 KiB
1 MiB	2 KiB	512	2 KiB
1 MiB	4 KiB	256	1 KiB

문제 2 · 할당 비율 -u

답 높일수록 유효 PTE가 많아져 무효 주소를 만날 가능성이 줄어든다. 시드 0의 결과는 아래와 같다.

해설 -u는 페이지별 할당 확률이다. 16개라는 짧은 표본에서 25%가 정확히 4페이지가 되는 것은 아니다. 테이블은 항상 16개 항목을 가져 크기는 64바이트로 같다. 변환은 VPN=floor(VA/1024), offset=VA mod 1024, PA=PFN×1024+offset으로 한다.

u(%)	유효 PTE/16	유효 참조/5	무효 참조/5
0	0	0	5
25	6	1	4
50	9	3	2
75	16	5	0
100	16	5	0

문제 3 · 세 가지 무작위 배치의 변환

답 원문에 지정된 시드·크기·기본 u=50을 사용한 각 VA의 결과는 아래와 같다.

해설 ① 8바이트 페이지·32바이트 주소 공간은 구조를 보여 주는 극단적인 장난감 예다. 너무 작은 페이지는 테이블·TLB 관리 부담이 크다. ② 8 KiB, ③ 1 MiB는 더 큰 페이지의 절충을 관찰하는 예이며, 1 MiB라는

크기만으로 주소 변환 원리상 불가능한 것은 아니다. 실제 지원 크기는 아키텍처에 따른다.

조건	VA	PA 또는 무효 이유
1	0x0000000e	0000030e (decimal 782) [VPN 1]
1	0x00000014	Invalid (VPN 2 not valid)
1	0x00000019	Invalid (VPN 3 not valid)
1	0x00000003	Invalid (VPN 0 not valid)
1	0x00000000	Invalid (VPN 0 not valid)
2	0x000055b9	Invalid (VPN 2 not valid)
2	0x00002771	Invalid (VPN 1 not valid)
2	0x00004d8f	Invalid (VPN 2 not valid)
2	0x00004dab	Invalid (VPN 2 not valid)
2	0x00004a64	Invalid (VPN 2 not valid)
3	0x0308b24d	1f68b24d (decimal 526955085) [VPN 48]
3	0x042351e6	Invalid (VPN 66 not valid)
3	0x02feb67b	0a9eb67b (decimal 178173563) [VPN 47]
3	0x0b46977d	Invalid (VPN 180 not valid)
3	0x0dbcceb4	1f2cceb4 (decimal 523030196) [VPN 219]

문제 4 · 시뮬레이터의 제약

답 이 선형 변환 시뮬레이터는 물리 메모리가 주소 공간보다 커야 한다. A=64 KiB, P_phys=32 KiB는 u=25라도 설정 오류로 끝난다.

해설 이것은 도구의 단순화 제약이다. 실제 가상 메모리 시스템은 부분 상주·스왑 등으로 가상 공간이 RAM보다 클 수 있다. 페이지·주소 공간·물리 메모리의 크기는 도구의 2의 거듭제곱 및 정수 페이지 배치 조건을 만족해야 한다. 예: 페이지 3 KiB와 주소 공간 16 KiB는 정수배가 아니어서 거부된다.

19장 / 핵심 요약

TLB: 더 빠른 주소 변환

19장 | 자주 쓰는 VPN → PFN 매핑을 CPU 가까이에 저장

> **TLB(Translation Lookaside Buffer)**는 주소 변환 정보의 캐시다.
> 사용자 데이터 자체를 저장하는 데이터 캐시와 역할이 다르다.

상황	동작
TLB hit	권한을 확인하고 PFN과 offset을 합쳐 물리 주소 구성
TLB miss, 페이지는 RAM에 있음	페이지 테이블 검색 → 변환을 TLB에 저장 → 접근 재시도
페이지가 RAM에 없음	페이지 폴트 → 운영체제가 반입·할당 등 처리

지역성과 TLB 도달 범위

- **공간 지역성**: 같은 페이지의 연속 배열 원소는 처음 한 번의 미스 후 같은 변환을 재사용한다.
- **시간 지역성**: 가까운 시점에 같은 페이지를 다시 사용하면 기존 변환을 재사용한다.

> **TLB 도달 범위(reach) = 엔트리 수 × 페이지 크기**
> **64개 × 4 KiB = 256 KiB / 64개 × 2 MiB = 128 MiB**

도달 범위는 같은 페이지 크기의 엔트리와 충분한 연관도를 가정한 용량 계산이다. 실제 히트율은 접근 패턴과 충돌에도 좌우된다.

미스 처리와 문맥 교환

하드웨어 관리 TLB는 하드웨어가 테이블을 탐색한다(교재의 x86 예). 소프트웨어 관리 TLB는 운영체제 핸들러가 변환을 넣는다. 미스 처리 후에는 원래 접근을 다시 실행해야 한다.

프로세스가 바뀌면 같은 VPN이 다른 PFN을 뜻할 수 있다. **TLB 비우기**로 이전 변환을 무효화하거나, **ASID**로 주소 공간을 구분해 여러 프로세스의 변환을 함께 보관한다.

비트 구분: TLB의 valid는 캐시 항목의 유효성이다. PTE의 valid는 주소 공간 내 매핑의 유효성이다. TLB가 비었다고 페이지 자체가 잘못된 주소인 것은 아니다.

숙제 (측정)

이 숙제에서는 TLB를 접근하는 크기와 비용을 측정할 것이다. 이 개념은 Saavedra-Barrera의 연구[Saa92]에 기초한 것으로 이들이 만든 간단하지만 캐시 계층의 여러 측면을 측정하는 아름다운 기법을 이용한다. 그리고 이 모든 것이 아주 간단한 사용자 수준의 프로그램을 통해 할 수 있다. 좀 더 자세히 알고 싶다면 이 연구를 읽어보도록 하자.

기본 개념은 커다란 자료 구조(예, 배열) 내에서 몇 개의 페이지들을 접근하고 그 접근들에 대한 시간을 측정하는 것이다. 예를 들어서 어떤 기계의 TLB 크기는 4라고 해보자(매우 작기는 하겠지만 이 논의의 목적에 충분히 부합한다). 프로그램을 작성하였는데, 그 프로그램이 네 개 또는 그보다 적은 페이지들을 접근한다면 각 접근들은 TLB 히트가 될 것이며 상대적으로 빠를 것이다. 하지만, 다섯 개 또는 그 이상을 반복문에서 계속 접근한다면 TLB 미스를 만들어서 각 접근은 비용이 갑자기 크게 늘어나게 된다.

다음의 간단한 코드에서 배열을 한 번만 접근하는 반복문을 표현하였다.

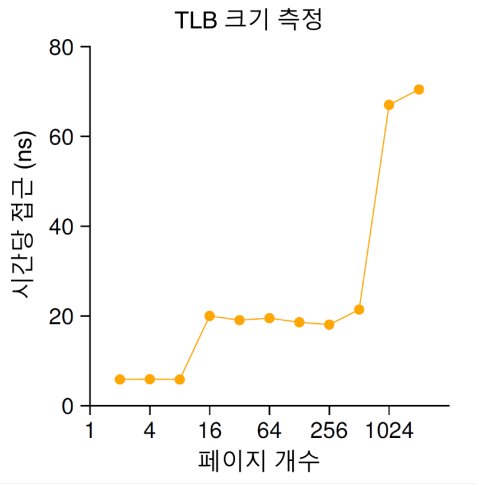
```
int jump = PAGESIZE / sizeof(int);
for (i = 0; i < NUMPAGES * jump; i += jump) {
    a[i] += 1;
}
```

이 반복문에서는 배열 `a`는 페이지당 하나의 정수 형 크기가 갱신이 되며 정해진 `NUMPAGES` 개만큼의 페이지들이 변경된다. 여러 차례 이 반복문이 수행되는 것(이 반복문을 반복하는 또 다른 반복문을 1억 번 수행하거나 또는 이 반복문이 수초 동안 실행될 수 있을 만큼의 반복을 수행한다)을 측정하는 것을 통해 각 접근이 얼마나 오랜 시간이 걸리는지 (평균적으로) 측정할 수 있다. `NUMPAGES`가 커짐에 따라서 비용이 뛰는 정도를 살펴보면 일 단계 TLB의 크기가 얼마나 큰지를 알 수 있으며 이 단계 TLB가 존재하는지를 판단할 수 있으며(존재한다면 얼마나 큰지도 함께), 그리고 TLB 히트와 미스가 성능에 어떤 영향을 줄 수 있는지에 대한 감을 잡을 수가 있다.

그림 22.5에 예시 그래프가 있다. 그래프에서 볼 수 있듯이, 몇 개의 페이지들을 접근할 때(8개 또는 그 이하) 평균 접근 시간은 약 5 nsec가 걸렸다. 16개 또는 그 이상의 페이지들이 접근되었을 때는 접근마다 약 20 nsec로 갑자기 비용이 늘어났다. 그 다음으로 비용이 늘어나는 구간은 1024개 이상의 페이지들을 접근할 때이며 각 접근은 약 70 nsec가 소요된다. 이 데이터에서 이 단계 TLB 계층이 있다고 결론지을 수 있으며 첫 번째는 상당히 작다(8개에서 16개의 항목을 담을 수 있는 것으로 보임). 두 번째는 좀 더 크지만 더 느리다(약 512개를 담을 수 있음). 일 단계 TLB의 히트들과 미스들을 비교해 보면 그 차이가 상당한 것을 알 수 있으며 약 14배 정도 차이나는 것을 볼 수 있다. TLB 성능은 중요하다!

문제

1. 시간을 재기위해서는 `gettimeofday()`를 통해서 만들 수 있는 타이머를 사용해야 할 것이다. 이 타이머는 얼마나 정확한가? 시간을 정확하게 재기 위해서는 타이머가



〈그림 22.5〉 **TLB 크기와 미스의 비용 파악하기**

얼마나 길어야 하는가?(반복문 내에서 몇 번이나 페이지를 접근해야 시간을 정확히 잴 수 있는지를 결정하는 데 도움을 줄 것이다)

2. `tlb.c` 라는 프로그램을 작성하여 각 페이지를 접근하는 데 드는 대략적인 비용을 측정해 보자. 프로그램의 입력 값은 접근해야 하는 페이지들의 수와 시도 횟수이다.

3. 선호하는 스크립트 언어(csh, python 등)를 사용하여 프로그램을 실행시키는 스크립트를 작성하여 접근하는 페이지들의 수를 1부터 수천까지 2의 배수로 증가시키도록 해 보자. 다른 기계들에서 스크립트를 실행하여 데이터를 수집하자. 신뢰할 수 있는 측정치를 얻기 위해서는 몇 번이나 시도해 봐야 하는가?

4. 그 다음으로 결과들을 사용하여 앞서 보인 그래프처럼 만들어 보자. `ploticus`와 같은 툴을 사용하면 된다. 시각화하는 것은 데이터를 좀 더 잘 이해하는 데 도움이 되는데, 왜 그런 것 같은가?

5. 한 가지 주의해야 할 것은 컴파일러 최적화이다. 컴파일러는 프로그램의 다른 부분들이 사용하지 않는 변수를 반복문을 통해 증가시키는 부분을 빼버리는 등의 온갖 교묘한 짓들을 한다. 컴파일러가 작성한 TLB 크기 추정기를 어떻게 수정하여야 컴파일러가 주 반복문을 제거하지 않도록 만들 수 있을까?

6. 또 한 가지 주의해야 할 것은 대부분의 시스템은 CPU가 여러 개이며, 각 CPU는 물론 각각의 TLB 계층을 갖고 있다. 정말 제대로 측정하기 위해서는 스케줄러가 하나의 CPU에서 또 다른 CPU로 이동시키지 못하도록 만들어서 하나의 CPU만을 사용해서 측정해야 할 것이다. 어떻게 그렇게 할 수 있을까? (힌트: 구글에서 "쓰레드 고정하기 (pinning a thread)"를 검색해 보자) 이렇게 하지 않았을 때는 어떤 일이 발생할까? 그리고 코드가 한 CPU에서 다른 CPU로 이동할 때 어떤 일이 발생할까?

7. 또 다른 발생할 수 있는 문제는 초기화와 관련되어 있다. 접근하기 전에 배열 `a`를 초기화하지 않는다면, 처음 `a`의 배열을 접근할 경우 한 번도 접근한 적이 없기 때문에

매우 비싼 비용이 들 것이다. 이 부분이 시간을 재는 데에 영향을 줄까? 이러한 잠재적인 비용 요인의 균형을 잡기 위해서 무엇을 할 수 있을까?

19장 / 정답 · 해설

TLB

코드는 Linux/POSIX용 예시다. 이 Windows 작업 환경에서 Linux 하드웨어 수치를 실측하지 않았다. 실측 문항은 방법·해석·예상 경향을 답으로 제시한다.

문제 1 · 타이머 정확도와 측정 길이

답 환경마다 다르다. 타이머 분해능을 먼저 확인하고, 총 측정 시간이 오차·조회 비용보다 충분히 길도록 반복 수를 늘린다.

해설 gettimeofday는 μs를 출력하지만 그 자체가 정확도 보장은 아니다. 예컨대 전체 구간을 100 ms 이상으로 늘리고 반복 측정의 중앙값·변동폭을 비교할 수 있다. 100 ms는 보편적인 정답 상수가 아니라 재현을 위한 시작점이다. CPU 주파수·부하·가상 머신도 기록한다.

문제 2 · tlb.c: 페이지당 하나씩 접근

답 페이지 수와 반복 횟수를 인자로 받고, 페이지마다 한 바이트를 갱신해 평균 ns/access를 계산한다.

해설 메모리는 측정 전에 처음 접근하고 워밍업한다. 아래 gettimeofday 기반 예에서는 총 구간을 충분히 길게 해야 한다. volatile 접근으로 최적화 제거를 막았다. 데이터 캐시의 충돌·대형 페이지 등도 지연에 영향을 줄 수 있으므로 관찰된 계단을 TLB 크기만으로 단정하지 않는다.

gcc -O2 tlb.c -o tlb / ./tlb 64 100000

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <unistd.h>
#include <sys/time.h>
static double ns(void) {
    struct timeval t; gettimeofday(&t,NULL);
    return t.tv_sec*1e9+t.tv_usec*1000.0;
}
int main(int argc,char **argv) {
    if(argc!=3) return 1;
    size_t pages=strtoull(argv[1],NULL,10);
    size_t reps=strtoull(argv[2],NULL,10);
    long psz=sysconf(_SC_PAGESIZE);
    if(!pages || !reps || psz<=0 || pages>SIZE_MAX/(size_t)psz)
        return 1;
    size_t step=(size_t)psz, bytes=pages*step;
    unsigned char *p=calloc(1,bytes);
    if(!p) return 1;
    volatile unsigned char *a=p;
    for(size_t j=0;j<pages;j++) a[j*step]=1;
    for(int warm=0;warm<10;warm++)
        for(size_t j=0;j<pages;j++) a[j*step]++;
    double t=ns();
    for(size_t r=0;r<reps;r++)
        for(size_t j=0;j<pages;j++) a[j*step]++;
    double elapsed=ns()-t;
    printf("%zu,%zu,%.3f\n",pages,reps,
           elapsed/((double)pages*reps));
    free(p); return 0;
}
```

문제 3 · 페이지 수·반복 횟수 바꾸기

답 페이지 수를 1,2,4,...,4096으로 바꾸고 각 조건을 여러 번 실행한다. 고정된 최소 시행 횟수의 정답은 없다.

해설 예시 스크립트는 20회 시행해 중앙값을 계산할 수 있는 CSV를 만든다. 반복 횟수는 한 번의 측정이 충분히 긴지 확인한 뒤 조정한다. 중앙값과 IQR·오차 막대가 안정되는지도 본다. 페이지 수가 늘면 같은 reps의 총 실행시간도 늘므로 모든 조건의 정밀도가 같다고 가정하지 않는다.

python3 sweep-tlb.py > tlb.csv

```
import subprocess
print("pages,reps,ns_access")
for power in range(13):
    pages=2**power
    for trial in range(20):
        line=subprocess.check_output(
            ["./tlb",str(pages),"100000"],text=True)
        print(line.strip())
```

문제 4 · 그래프로 확인하기

답 x축은 페이지 수(log2), y축은 페이지당 접근 시간(ns)으로 두고 중앙값과 변동폭을 그린다.

해설 작은 페이지 수에서 낮은 지연, 특정 범위를 넘어선 지연 상승이 보이면 캐시·TLB 용량과 연관해 분석할 수 있다. 아래는 실제로 저장한 CSV를 그리는 코드다. 이 Windows 환경에서 얻은 사용자 Linux 하드웨어 측정값은 제공하지 않는다.

실측 CSV용 그래프 템플릿; 측정 결과를 만들어 넣는 코드가 아니다

```
import csv,statistics
import matplotlib.pyplot as plt
groups={}
with open("tlb.csv") as f:
    for r in csv.DictReader(f):
        groups.setdefault(int(r["pages"]),[]).append(float(r["ns_access"]))
x=sorted(groups)
y=[statistics.median(groups[n]) for n in x]
plt.plot(x,y,"o-"); plt.xscale("log",base=2)
plt.xlabel("Pages"); plt.ylabel("ns/access")
plt.grid(True); plt.savefig("tlb-measured.png",dpi=200)
```

문제 5 · 최적화에 의한 반복 제거 방지

답 volatile 메모리 접근 또는 결과를 관찰 가능한 형태로 사용해 반복문이 살아 있도록 한다.

해설 2번 예는 volatile 포인터로 쓰기를 수행한다. 단순히 지역 변수를 증가시키고 한 번도 쓰지 않으면 컴파일러가 루프를 제거할 수 있다. 최적화 옵션만 낮추면 다른 실행 비용도 바뀌므로 측정 목적에 맞는 코드를 먼저 만든다.

문제 6 · 한 CPU에 고정하기

답 Linux의 sched_setaffinity 또는 taskset으로 허용된 CPU 하나에 고정한다.

해설 먼저 taskset -pc PID나 시스템의 허용 CPU 목록을 확인한다. 예: taskset -c <허용된 CPU 번호> python3 sweep-tlb.py를 실행하면 자식도 같은 친화성을 물려받는다. 이동이 생기면 각 CPU의 TLB·캐시가 다시 채워지고 잡음이 증가한다. NUMA 메모리 위치와 다른 부하도 통제한다.

문제 7 · 초기화·첫 접근 비용

답 처음 접근하는 페이지의 요구 할당·0 초기화·페이지 폴트가 TLB 미스 비용과 섞인다. 측정 전에 모든 페이지를 쓰고 워밍업한다.

해설 calloc 호출만으로 물리 페이지가 모두 상주했다고 단정할 수 없다. 2번의 페이지별 쓰기는 실물 페이지를 준비하는 데 도움이 된다. 첫 실행(cold)과 워밍업 뒤(steady-state)를 구분해서 보고하면 목적에 맞는 비교가 된다. 워밍업으로 전체가 TLB에 들어간다는 보장은 없다.

20장 / 핵심 요약

더 작은 페이지 테이블

20장 | 희소한 주소 공간을 큰 배열 전체로 표현하지 않는다

방법	공간 절약 방식	대가
큰 페이지	페이지 수와 PTE 수 감소	내부 단편화 증가 가능
세그먼트+페이징	코드·힙·스택별 필요한 테이블만 유지	영역 관리가 복잡하고 테이블 할당의 문제가 남음
다단계 테이블	하위 매핑이 없는 테이블 페이지는 생성하지 않음	TLB 미스 때 여러 단계 탐색
역 페이지 테이블	시스템의 물리 프레임당 항목 하나	프로세스+VPN 검색이 필요해 해시 등 사용

다단계 테이블의 핵심

VPN을 여러 인덱스로 나눈다. 페이지 디렉터리의 항목이 다음 테이블 페이지를 가리키고, 마지막 PTE가 데이터의 PFN을 제공한다. 미사용 주소 영역에는 하위 테이블을 두지 않는다.

> **32비트·4 KiB·PTE/PDE 4바이트의 2단계 예**
> **VA = 디렉터리 인덱스 10비트 | 테이블 인덱스 10비트 | offset 12비트**

테이블 한 페이지에는 4096/4=1024개 PTE가 들어간다. 최상위 디렉터리는 4 KiB다. 하위 테이블 2개만 필요하면 전체는 **12 KiB**이고, 모든 하위 테이블이 필요하면 약 **4 MiB+4 KiB**다.

시간과 공간의 절충

- TLB 히트에서는 다단계 탐색을 건너뛴다. 공간 절약의 대가는 주로 TLB 미스 때 나타난다.
- n단계 테이블이면 단순 모델에서 TLB 미스 시 테이블 n번과 데이터 1번의 접근이 필요하다. 캐시 효과는 별도로 고려한다.
- 페이지 단위로 테이블을 할당하므로 거대한 연속 물리 공간을 찾을 필요가 없다. 사용 범위가 작을 때 특히 유리하다.

사용자 페이지 테이블 자체를 커널 가상 공간에 배치하고 일부를 스왑하는 설계도 있다. 이 경우 탐색 과정이 더 복잡해진다.

숙제

멀티 레벨 페이지 테이블이 어떻게 동작하는지를 이해하고 있는지를 확인하기 위한 재미있는 작은 숙제이다. 앞 문장에서 "재미"있다는 표현이 적합한지에 대해서는 논란이 있는 것은 사실이다. 놀랍지 않겠지만, 프로그램의 이름은 `paging-multilevel-translate.py`이다. README의 상세한 설명을 참고하자.

문제

1. TLB 미스에 대해서 하드웨어가 검색을 한다는 가정 하에서 선형 페이지 테이블을 사용하면 페이지 테이블의 위치를 찾기 위해 하나의 레지스터가 필요하다. 2단계 페이지 테이블을 사용하는 경우에는 몇 개의 레지스터가 필요한가? 3단계 페이지 테이블을 사용하는 경우는 몇 개가 필요한가?
2. 시뮬레이터를 이용하여 변환을 수행하여 보자. 이때 랜덤 시드는 0과 1 그리고 2를 사용하여 보자. `-c` 플래그를 사용하여 답이 맞는지 확인해 보자. 각 검색을 수행하기 위해서 몇 번의 메모리 참조가 필요한가?
3. 캐시 메모리가 어떻게 동작하는지 이해했다고 했을 때, 캐시를 사용하면 페이지 테이블에 대한 메모리 참조는 어떻게 동작할까? 캐시 히트가 많이 생겨날까(그래서 빠르게 접근될 것인가)? 또는 많은 미스를 만들어 낼까(그러므로 느리게 접근될 것인가)?

20장 / 정답 · 해설

더 작은 페이지 테이블

시뮬레이션 수치는 교재 저자의 공식 프로그램과 명시한 조건으로 검산했다. 미지정 옵션은 해당 버전의 기본값을 사용한다.

문제 1 · 단계 수와 루트 레지스터 수

답 선형·2단계·3단계 모두 **최상위 테이블을 가리키는 루트 레지스터 1개**면 된다.

해설 하위 테이블의 위치는 상위 항목이 제공한다. 단계마다 전용 루트 레지스터가 추가되는 것은 아니다. 페이지 크기·형식이 정해져 있다는 전제이며, 전체 MMU에 레지스터가 딱 하나만 있다는 뜻은 아니다.

문제 2 · 시드 0·1·2 다단계 변환

답 기본 모델은 VA 15비트, 페이지 32바이트다. VA=PDE 인덱스 5비트 | PTE 인덱스 5비트 | offset 5비트다. 모든 주소의 검산 결과는 아래와 같다.

해설 예: 시드 0의 VA=0x611c는 PDE=24, PTE=8, offset=28이다. PTE PFN=53이므로 PA=53×32+28=0x6bc다. 마지막 열은 캐시·TLB 효과를 제외하고 실제 데이터까지 읽는 전체 메모리 참조 수다. 성공은 PDE+PTE+데이터로 3회, PDE 무효는 1회, PTE 무효는 2회다. 순수 변환의 테이블 접근만 세면 성공 시 2회다.

시드	VA	PDE	PTE	결과	참조
0	0x611c	0x18	0x8	0x6bc	3
0	0x3da8	0xf	0xd	PTE 무효	2
0	0x17f5	0x5	0x1f	0x9d5	3
0	0x7f6c	0x1f	0x1b	PTE 무효	2
0	0x0bad	0x2	0x1d	PTE 무효	2
0	0x6d60	0x1b	0xb	PTE 무효	2
0	0x2a5b	0xa	0x12	PTE 무효	2
0	0x4c5e	0x13	0x2	PTE 무효	2
0	0x2592	0x9	0xc	0x7b2	3
0	0x3e99	0xf	0x14	0x959	3
1	0x6c74	0x1b	0x3	0xc34	3
1	0x6b22	0x1a	0x19	0x8e2	3
1	0x03df	0x0	0x1e	0x0bf	3
1	0x69dc	0x1a	0xe	PTE 무효	2
1	0x317a	0xc	0xb	0x6ba	3

시드	VA	PDE	PTE	결과	참조
1	0x4546	0x11	0xa	PTE 무효	2
1	0x2c03	0xb	0x0	0xae3	3
1	0x7fd7	0x1f	0x1e	PTE 무효	2
1	0x390e	0xe	-	PDE 무효	1
1	0x748b	0x1d	0x4	PTE 무효	2
2	0x7570	0x1d	0xb	PTE 무효	2
2	0x7268	0x1c	0x13	0xca8	3
2	0x1f9f	0x7	0x1c	PTE 무효	2
2	0x0325	0x0	0x19	0xba5	3
2	0x64c4	0x19	0x6	PTE 무효	2
2	0x0cdf	0x3	0x6	0x2ff	3
2	0x2906	0xa	-	PDE 무효	1
2	0x7a36	0x1e	0x11	0xcd6	3
2	0x21e1	0x8	-	PDE 무효	1
2	0x5149	0x14	0xa	0x029	3

문제 3 · 캐시에 의한 테이블 접근 가속

답 같은 주소 영역을 반복 접근하면 상위 테이블과 인접 PTE에 캐시 히트가 생기기 쉽다. 무작위·넓은 접근은 불리할 수 있다.

해설 다수 VA가 같은 PDE·테이블 페이지를 공유해 상위 단계에 재사용이 많다. 한 캐시 라인에 여러 PTE가 들어가 공간 지역성도 생긴다. 실제 비용은 각 단계가 반드시 RAM까지 가는 n번의 고정 비용과 같지 않다. TLB 히트는 테이블 탐색 자체를 건너뛴다.

21장 / 핵심 요약

스왑과 페이지 폴트

21장 | RAM보다 큰 가상 메모리를 제공하는 메커니즘

RAM을 모든 가상 페이지 중 일부를 담는 캐시로 본다. 익명 페이지는 스왑 공간에 보관할 수 있고, 변경되지 않은 실행 파일·파일 매핑 페이지는 원본 파일에서 다시 가져올 수 있다.

메모리 접근의 분기

확인 결과	의미	후속 동작
주소가 무효	사용할 수 없는 가상 영역	오류 처리·시그널·종료 등
접근 권한 위반	읽기/쓰기/실행 방식이 허용되지 않음	보호 오류 처리
유효·허용·present=1	페이지가 RAM에 있음	변환을 TLB에 넣고 접근 재시도
유효·허용·present=0	합법적인 페이지가 RAM에 없음	페이지 폴트 핸들러가 필요한 페이지 준비

디스크에서 페이지를 가져오는 절차

1. 빈 물리 프레임을 확보한다.
2. 없으면 교체 정책으로 피해 페이지를 정한다.
3. 보존해야 하는 dirty 데이터는 저장 장치에 기록한다.
4. 요청 페이지를 읽는다. I/O 대기 중 다른 프로세스를 실행할 수 있다.
5. PTE의 PFN·present와 관련 변환을 갱신한다.
6. 폴트를 일으킨 명령어를 다시 실행한다.

미리 여유 메모리 확보하기

빈 프레임이 **low watermark** 아래로 내려가면 백그라운드 회수 작업을 시작하고, **high watermark**까지 확보하면 쉰다. 더티 페이지를 묶어 기록하는 클러스터링은 작은 I/O를 반복하는 비용을 줄인다.

이 장의 기본 폴트는 디스크 반입 모델이다. Demand zero·COW처럼 디스크 읽기 없이 처리하는 페이지 폴트도 있다. "폴트=프로그램 버그" 또는 "폴트=항상 디스크 I/O"로 외우지 않는다.

측정 실습: vmstat와 메모리 사용

추가 번역 | 영문 10-11쪽의 Homework (Measurement)

이 실습에서는 새로운 도구 vmstat를 소개하고, 이 도구로 메모리, CPU, I/O 사용량을 파악하는 방법을 알아본다. 아래 실습과 문제를 시작하기 전에 함께 제공되는 README를 읽고 mem.c의 코드를 살펴보자.

1. CPU 사용량 관찰

먼저 같은 컴퓨터에 연결된 터미널 창 두 개를 열어, 각 창에서 서로 다른 프로그램을 쉽게 실행할 수 있도록 하자. 한 창에서 vmstat 1을 실행한다. 이 명령은 컴퓨터의 사용 통계를 1초마다 보여 준다. 출력 내용을 이해할 수 있도록 매뉴얼 페이지, 함께 제공된 README, 그리고 필요한 다른 자료를 읽어 보자. 아래의 나머지 실습을 진행하는 동안 이 창에서 vmstat를 계속 실행해 둔다.

이제 mem.c 프로그램을 아주 적은 메모리만 사용하도록 실행하자. ./mem 1을 입력하면 1 MB의 메모리만 사용한다. mem을 실행할 때 CPU 사용 통계는 어떻게 달라지는가? 사용자 시간(user time) 열의 값은 타당한가? mem을 여러 개 동시에 실행하면 이 값은 어떻게 달라지는가?

2. 메모리 통계 관찰

mem을 실행하면서 메모리 통계를 살펴보자. 두 열에 집중한다. swpd(사용 중인 스왑 메모리의 양)와 free(유휴 메모리의 양)이다. ./mem 1024를 실행하여 1024 MB를 할당하고, 두 값이 어떻게 달라지는지 관찰한다. 그런 다음 Ctrl+C를 눌러 실행 중인 프로그램을 종료하고, 값이 어떻게 바뀌는지 다시 관찰한다. 무엇을 발견했는가? 특히 프로그램이 종료될 때 free 열은 어떻게 달라지는가? mem이 종료되면 여유 메모리가 예상한 만큼 늘어나는가?

3. 스왑 입출력 관찰

다음으로 스왑 열인 si와 so를 살펴보자. 이 열들은 디스크에서 메모리로, 그리고 메모리에서 디스크로 얼마나 많은 스와핑이 발생하는지 보여 준다. 이 값들이 변하게 하려면 mem을 많은 메모리를 사용하도록 실행해야 한다.

먼저 Linux 시스템에 여유 메모리가 얼마나 있는지 확인한다. 예를 들어 cat /proc/meminfo를 입력한다. /proc 파일 시스템과 이곳에서 확인할 수 있는 정보의 종류는 man proc으로 알아볼 수 있다. /proc/meminfo의 첫 항목 중 하나는 시스템의 전체 메모리 용량이다.

메모리가 약 8 GB라고 가정하자. 먼저 mem 4000(약 4 GB)를 실행하고 스왑 입출력 열을 관찰한다. 0이 아닌 값이 나타나는가? 다음으로 5000, 6000 등의 값을 차례로 사용해 보자. 첫 번째 루프와 비교하여 두 번째 루프와 그 이후의 루프에 들어갈 때 이 값들은 어떻게 달라지는가? 두 번째, 세 번째, 그리고 이후의 루프에서 스왑 인과 스왑 아웃되는 데이터의 총량은 각각 얼마인가? 이 수치들은 타당한가?

측정 실습: 문제 4-7

영문 추가 내용 한국어 번역 (계속)

4. CPU와 블록 I/O 통계 비교

앞에서와 같은 실험을 반복하되, 이번에는 CPU 사용률과 블록 I/O 통계 등 다른 통계를 관찰한다. `mem`이 실행되는 동안 이 값들은 어떻게 달라지는가?

5. 메모리에 들어가는 경우와 스와핑하는 경우의 성능

이제 성능을 살펴보자. `mem`의 입력으로 메모리에 충분히 들어갈 수 있는 값을 선택한다. 예를 들어 시스템 메모리가 8 GB라면 4000을 선택한다. 루프 0과 그 이후의 루프 1, 2 등은 각각 얼마나 오래 걸리는가?

이번에는 물리 메모리 용량을 충분히 초과하는 크기를 선택한다. 역시 메모리가 8 GB라고 가정하면 12000을 선택할 수 있다. 이 경우 각 루프는 얼마나 오래 걸리는가? 대역폭 수치는 어떻게 다른가? 지속적으로 스와핑할 때와 모든 데이터가 메모리에 충분히 들어갈 때의 성능은 얼마나 다른가?

`mem`이 사용하는 메모리의 크기를 x축으로, 그 메모리에 접근하는 대역폭을 y축으로 하는 그래프를 그릴 수 있는가? 마지막으로, 모든 데이터가 메모리에 들어가는 경우와 그렇지 않은 경우 각각에서 첫 번째 루프의 성능은 이후 루프들의 성능과 어떻게 다른가?

6. 스왑 공간의 한계

스왑 공간은 무한하지 않다. `swapon` 도구에 `-s` 옵션을 주면 사용 가능한 스왑 공간을 확인할 수 있다. 사용 가능한 스왑 용량을 넘어서는 것처럼 보일 정도로 점점 더 큰 값을 주어 `mem`을 실행하면 어떤 일이 발생하는가? 메모리 할당은 어느 시점에서 실패하는가?

7. 심화: 스왑 장치에 따른 성능

마지막으로, 심화 실습을 할 수 있다면 `swapon`과 `swapoff`로 시스템이 서로 다른 스왑 장치를 사용하도록 설정해 보자. 자세한 내용은 매뉴얼 페이지를 읽어 보자. 서로 다른 하드웨어를 사용할 수 있다면, 전통적인 하드 디스크, 플래시 기반 SSD, 심지어 RAID 배열을 스왑 장치로 사용할 때 스와핑 성능이 어떻게 달라지는지 확인하자. 새로운 장치로 스와핑 성능을 얼마나 향상시킬 수 있는가? 메모리에 직접 접근하는 성능에 얼마나 가까워질 수 있는가?

번역 메모: 문제 2의 원문은 swpd를 "사용 중인 가상 메모리의 양"이라고 표현한다. 이 번역에서는 vmstat의 열 이름에 맞추어 "사용 중인 스왑 메모리의 양"으로 명확히 했다. 8 GB와 각 입력값은 원문에서 제시한 예시이며, 실습 문제의 명령과 수치는 원문을 유지했다. swpd 정의 확인: procps-ng vmstat(8) 매뉴얼.

21장 / 정답 · 해설

스왑 메커니즘

코드는 Linux/POSIX용 예시다. 이 Windows 작업 환경에서 Linux 하드웨어 수치를 실측하지 않았다. 실측 문항은 방법·해석·예상 경향을 답으로 제시한다.

문제 1 · vmstat의 CPU 사용량

답 mem 1은 계산을 반복하므로 us가 증가한다. CPU가 N개인 시스템에서 한 단일 스레드가 한 코어를 꽉 쓰면 전체 CPU 기준 약 100/N%일 수 있다.

해설 여러 mem을 실행하면 코어 수까지 계산량이 분산되고 그 뒤에는 CPU를 공유한다. vmstat 1의 첫 CPU 보고는 부팅 이후 평균이므로 이후 구간을 보거나 -y로 첫 보고를 생략한다. 한 프로세스의 top 수치와 전체 CPU 평균을 같은 분모로 읽지 않는다.

문제 2 · swpd·free와 프로그램 종료

답 실제로 1024 MiB를 접근하면 여유 메모리가 대체로 줄고 종료 후 회수된다. swpd는 사용 중인 스왑 용량이다.

해설 RAM 여유가 충분하면 swpd가 그대로일 수 있다. free는 사용하지 않는 RAM이고 캐시 회수·다른 작업·커널 비용 때문에 요청량과 변화량이 딱 맞지는 않는다. available도 함께 본다. 종료한 프로그램의 스왑은 회수되지만 다른 작업의 스왑 상태가 즉시 모두 사라질 필요는 없다.

문제 3 · si·so와 RAM을 넘어선 작업 집합

답 si는 초당 스왑 반입량, so는 초당 스왑 반출량이다. 작업 집합이 상주 가능한 RAM에 들어가면 워밍업 뒤 둘 다 거의 0일 수 있다.

해설 지속적으로 넘치면 재접근할 페이지를 다시 읽고 다른 페이지를 내보내야 해 두 값이 반복적으로 커진다. 사용 중인 RAM·캐시·다른 프로세스·회수 정책에 따라 경계가 달라져 8 GiB 시스템에서 반드시 5000부터 스왑한다고 답할 수는 없다. 구간별 si·so에 시간 간격을 곱해 합산하면 누적량을 추정한다. swpd는 용량, si/so는 전송률이다.

문제 4 · CPU·블록 I/O의 변화

답 RAM에 들어가면 계산 중심이고, 스왑 압박이 커지면 블록 I/O와 대기·커널 처리의 비중이 증가할 수 있다.

해설 bi/bo는 블록 장치 전송률이며 si/so의 스왑만을 뜻하지 않는다. us·sy·wa·id와 함께 패턴을 확인한다. SSD·다른 실행 작업에 따라 CPU가 I/O를 기다리는 모습이 달라지므로 하나의 고정 퍼센트를 정답으로 쓰지 않는다.

문제 5 · 첫 루프·상주 루프·스와핑 루프의 성능

답 대역폭=접근한 MiB/걸린 초다. 모든 데이터가 RAM에 있으면 이후 루프가 보통 안정되고, 계속 스와핑하면 시간 증가·대역폭 저하가 나타난다.

해설 첫 루프에는 페이지 준비·초기화 비용이 섞인다. 순환 스캔의 작업 집합이 RAM보다 크면 이후 루프도 지속적으로 반입·반출할 수 있다. 크기별 첫 루프와 이후 루프의 중앙값을 따로 그린다. 제공 mem.c는 미초

기화 int 증가와 장기 실행의 signed overflow를 주의해야 하므로, 재현 코드는 13장처럼 초기화된 unsigned·volatile 접근으로 구성하면 해석이 명확하다. 실제 ms·MiB/s는 이 문서에서 실측한 값이 없다.

13장 memory-user.c의 반복 부분을 바꾸는 측정 예시

```
/* with initialized volatile unsigned char *a and size n */
for (int loop=0;loop<10;loop++) {
    double begin=now();
    for(size_t i=0;i<n;i++) a[i]=(unsigned char)(a[i]+1);
    double sec=now()-begin;
    printf("%d,%.6f,%.3f\n",loop,sec,
           (n/(1024.0*1024.0))/sec);
}
```

문제 6 · 스왑 공간의 한계와 할당 실패

답 swapon --show 또는 swapon -s로 확인한다. 한계에 접근하면 할당 실패 또는 실행 중 메모리 부족 종료 등이 발생할 수 있다.

해설 malloc 실패 시점은 단순히 RAM+swap 합계와 같지 않다. overcommit 정책, 프로세스·컨테이너 제한, 다른 작업, 예약량에 의존한다. malloc이 성공해도 페이지를 실제로 쓸 때 OOM이 발생할 수 있다. mem 크기를 끝없이 키워야만 개념을 이해할 수 있는 것은 아니며, 이 문서는 사용자 기기의 메모리를 고갈시키는 실험을 수행하지 않았다.

문제 7 · HDD·SSD·RAID 스왑 비교

답 같은 작업 집합·RAM·CPU·요청 패턴에서 장치별 루프 시간과 유효 대역폭을 비교한다. 정확한 배수는 실제 측정으로 정해야 한다.

해설 SSD는 HDD의 탐색·회전 대기보다 빠른 접근을 제공할 수 있고 RAID는 구성·요청 병렬성에 따라 달라진다. 스왑에는 폴트·회수·I/O·스케줄링 비용도 들어가므로 순수 장치 대역폭만으로 성능을 결정하지 않는다. swapon/swapoff를 통한 장치 변경은 별도의 시스템 실습 단계다. 이 해설에서는 사용자의 스왑 구성을 바꾸지 않았다.

22장 / 핵심 요약

페이지 교체 정책

22장 + 보충 | 무엇을 남기고 무엇을 내보낼지 결정

정책	교체 대상	핵심 성질
OPT / MIN	다음 사용이 가장 먼 페이지	미래를 알아야 함. 최소 미스의 비교 기준
FIFO	가장 먼저 들어온 페이지	단순. 프레임을 늘려도 미스가 늘 수 있음
Random	무작위 페이지	이력 유지 비용이 작으나 실행마다 결과 변동
LRU	가장 오래 사용하지 않은 페이지	시간 지역성 이용. 정확한 접근 이력 유지가 비쌈
Clock	참조 비트가 0인 후보	참조 비트로 LRU를 싸게 근사

Clock의 동작

시계 바늘이 가리킨 페이지의 참조 비트가 1이면 0으로 지우고 다음으로 이동한다. 0이면 교체 후보로 선택한다. 쓰기 비용까지 고려하면 최근에 사용하지 않은 **clean 페이지**를 우선할 수 있다.

자주 출제되는 함정

- **Belady 이상 현상**: FIFO에서 프레임 3→4로 늘려도 폴트가 증가할 수 있다. LRU·OPT는 스택 성질을 가져 같은 문제가 없다.
- **순환 스캔**: 캐시보다 조금 큰 범위를 반복하면 LRU도 불리하다. 한 번 훑는 페이지가 자주 재사용하는 페이지를 밀어내지 않도록 탐색 내성(scan resistance)이 필요하다.
- **쓰래싱**: 활발히 쓰는 작업 집합이 RAM을 초과해 계산보다 페이지 반입·반출에 많은 시간을 쓰는 상태다.

> 쓰래싱 대처: 동시에 실행할 작업 수 감소, 작업 집합 고려, 메모리 확보.
> 요구 페이징·선반입(prefetching)·쓰기 묶음도 성능을 좌우한다.

저장 장치의 비용은 낮은 미스율에서도 큰 영향을 준다. 원본 보충의 개정 AMAT 식과 검산은 이 통합본 부록 「핵심 공식과 검산」을 참조한다.

숙제

`paging-policy.py`라는 시뮬레이터는 여러 페이지 교체 정책들을 대상으로 실험해 볼 수 있게 한다. 자세한 내용은 README를 읽어보자.

문제

1. 다음 인자를 주고 랜덤 주소를 생성하라: `-s 0 -n 10`, `-s 1 -n 10`, 그리고 `-s 2 -n 10`. FIFO 정책에서 LRU 정책으로 그리고 OPT 정책으로 변경하라. 생성된 주소 흐름의 모든 주소에 대해 히트인지 미스인지를 판별하라.
2. 캐시 크기가 5인 경우에 대해서 FIFO, LRU 그리고 MRU 정책을 사용할 때 최악의 주소 참조 스트림을 생성하라(최악의 참조 스트림은 가능한 최대 개수의 미스를 발생시킨다). 최악의 참조 스트림의 경우 성능을 극적으로 향상시켜 OPT의 성능에 근접하게 만들려면 캐시의 크기는 얼마나 커야 하는가?
3. 랜덤한 추적 기록을 생성하라(python 또는 perl을 사용하라). 이런 추적 기록을 적용할 때 여러 정책이 어떻게 동작할 것이라고 예상하는가?
4. 얼마간 지역성을 보이는 추적 기록를 생성하라. 그런 추적 기록을 어떻게 생성할 수 있을까? 생성된 추적 기록대로 메모리 참조가 일어날 때 LRU의 성능은 어떤가? RAND는 LRU에 비해서 얼마나 더 좋은 성능을 보이는가? CLOCK은 어떤 성능을 보이는가? 시계 비트의 수를 변경하였을 때 CLOCK은 어떤 성능을 보이는가?
5. Valgrind와 같은 프로그램을 사용하여 실제 응용 프로그램으로부터 정보를 추출하여 가상 페이지 참조 스트림을 생성하라. 예를 들어, `valgrind --tool=lackey --trace-mem=yes ls` 명령어를 실행시키면 `ls` 프로그램에 의해서 참조된 모든 명령과 데이터에 대한 거의 완벽한 참조 추적 기록을 출력할 것이다. 이 정보를 시뮬레이터에서 사용하기 위해서는 먼저 가상 메모리 참조를 가상 페이지 번호 참조로 변환해야 한다. 가상 주소의 오프셋 부분을 제거한 후(masking off), 오프셋 비트 수만큼 오른쪽으로 시프트하면 된다. 당신의 응용 프로그램의 경우, 참조 요청의 대부분을 만족시키는 캐시의 크기는 얼마인가? 캐시의 크기가 증가할 때 응용 프로그램의 작업 집합의 변화를 그래프로 나타내어라.

22장 / 정답 · 해설

페이지 교체 정책

시뮬레이션 수치는 교재 저자의 공식 프로그램과 명시한 조건으로 검산했다. 미지정 옵션은 해당 버전의 기본값을 사용한다.

문제 1 · FIFO·LRU·OPT의 전체 히트/미스열

답 기본 캐시 크기 3, 주소 번호 0-9, 처음 빈 캐시다. H=히트, M=미스이며 각 열은 같은 위치의 주소에 대응한다.

해설 FIFO는 삽입 순서, LRU는 최근 사용 순서, OPT는 미래의 다음 사용 시점으로 피해 페이지를 선택한다. 처음 읽는 페이지는 cold miss다. 이 10개짜리 예에서 시드 2는 세 정책의 히트 수가 우연히 같다.

시드	정책	주소열	H/M열	히트
0	FIFO	8 7 4 2 5 4 7 3 4 5	M M M M M H M M M M	1
0	LRU	8 7 4 2 5 4 7 3 4 5	M M M M M H M M H M	2
0	OPT	8 7 4 2 5 4 7 3 4 5	M M M M M H H M H H	4
1	FIFO	1 8 7 2 4 4 6 7 0 0	M M M M M H M M M H	2
1	LRU	1 8 7 2 4 4 6 7 0 0	M M M M M H M M M H	2
1	OPT	1 8 7 2 4 4 6 7 0 0	M M M M M H M H M H	3
2	FIFO	9 9 0 0 8 7 6 3 6 6	M H M H M M M M H H	4
2	LRU	9 9 0 0 8 7 6 3 6 6	M H M H M M M M H H	4
2	OPT	9 9 0 0 8 7 6 3 6 6	M H M H M M M M H H	4

문제 2 · 캐시 5의 최악 참조열

답 FIFO·LRU: 0,1,2,3,4,5를 반복하면 매번 미스다. MRU: 0,1,2,3,4를 채운 뒤 5,4,5,4…를 반복하면 매번 미스다.

해설 FIFO·LRU는 6개 순환 작업 집합이 5개 프레임을 계속 밀어낸다. MRU는 가장 최근 페이지 4를 5가 내보내고, 4가 다시 5를 내보낸다. 세 경우 모두 캐시를 6으로 키우면 6개 페이지의 최초 미스 뒤 반복 접근은 히트여서 OPT와 같다. 일반 워크로드의 필요 크기를 항상 6이라고 할 수는 없다.

문제 3 · 지역성 없는 무작위 참조

답 독립·균등한 M개 페이지와 크기 C의 가득 찬 캐시라면 과거 이력만 쓰는 정책의 기대 히트율은 C/M이다.

해설 0-9 중 균등한 주소 1000개, 캐시 5에서 구현 가능한 정책은 약 50%에 모인다. OPT는 미래 정보를 사용하므로 같은 한계를 그대로 적용하지 않으며 약 72%를 얻었다. 원문 보충 255쪽의 "현실적으로 구현 가능한 정책"이라는 한정이 중요하다.

정책	시드0	시드1	시드2	평균
FIFO	49.50%	49.00%	47.90%	48.80%
LRU	50.50%	49.80%	48.50%	49.60%
RAND	49.20%	51.80%	48.60%	49.87%
CLOCK	48.90%	48.90%	48.70%	48.83%
OPT	71.80%	72.30%	71.60%	71.90%

문제 4 · 지역성·CLOCK 카운터 실험

답 시드 2026으로 90%는 페이지 0-2, 10%는 3-19를 고른 3000개 참조, 캐시 5를 사용했다. 검산 결과는 아래와 같다.

해설 이 추적에서는 LRU가 RAND보다 약 6.2%p 높다. 첨부의 "RAND가 LRU보다 얼마나 좋은가"는 보충 256쪽에서 비교 방향을 고친 부분이다. CLOCK의 -b는 이 도구에서 참조 카운터의 최대값이지 b비트 하드웨어 필드라는 뜻이 아니다. 카운터가 크면 인기 페이지를 더 오래 보호할 수 있지만 항상 더 좋은 것은 아니다.

정책	b	히트	미스	히트율
FIFO	-	2512	488	83.73%
LRU	-	2706	294	90.20%
RAND	-	2520	480	84.00%
CLOCK	1	2569	431	85.63%
CLOCK	2	2645	355	88.17%
CLOCK	4	2707	293	90.23%
CLOCK	8	2739	261	91.30%
OPT	-	2777	223	92.57%

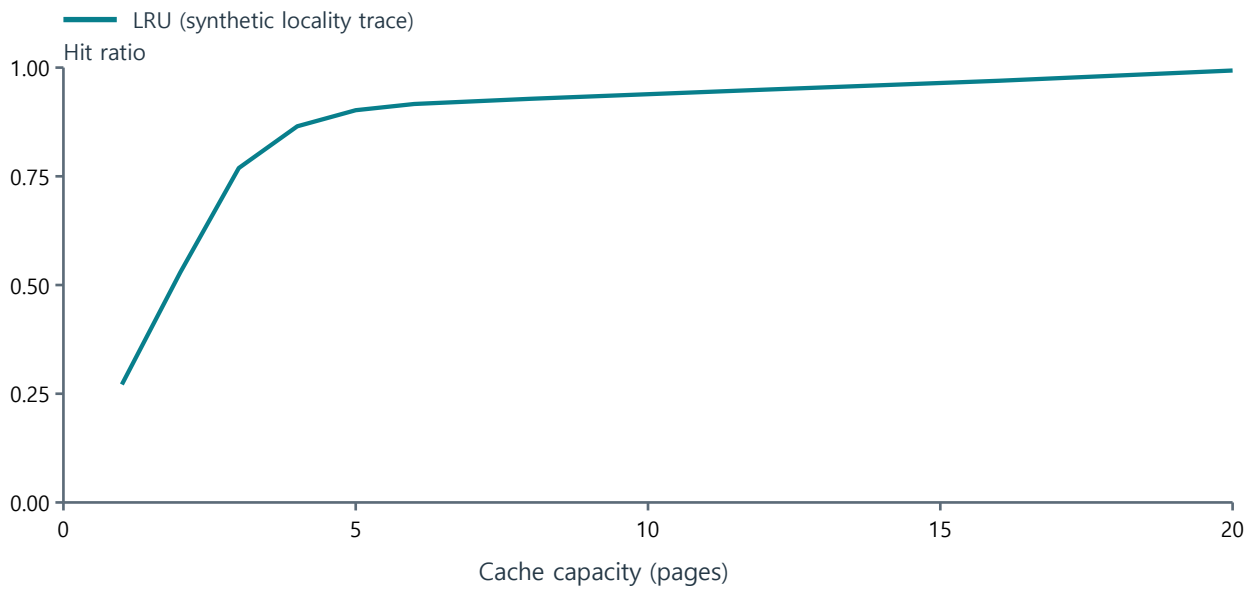
문제 5 · 실제 응용 프로그램 참조열과 작업 집합

답 Lackey의 메모리 추적을 페이지 번호로 바꿔 캐시 크기별 히트율을 그린다. 실제 앱의 필요한 크기는 해당 참조열과 목표 히트율에 따라 정한다.

해설 4 KiB 페이지이면 VPN=floor(VA/4096)=VA>>12다. 경계를 넘는 접근은 닿는 모든 페이지를 포함한다. 아래 파서는 I/L/S/M을 처리하고 M을 읽기+쓰기로 두 번 기록한다. 생성 파일을 paging-policy.py -f pages.txt -C 크기 -p LRU -c에 넣는다. 그래프는 이 문서에서 만든 합성 지역성 참조열의 예이며 실제 ls 측정 결과가 아니다.

Lackey 참조를 가상 페이지 번호로 변환

```
# valgrind --tool=lackey --trace-mem=yes ls 2> trace.txt
import re
with open("trace.txt") as f, open("pages.txt","w") as out:
    for line in f:
        m=re.search(r"\b([ILSM])\s+([0-9a-fA-F]+),(\d+)",line)
        if not m: continue
        op,addr,size=m.groups(); addr=int(addr,16); size=int(size)
        if size==0: continue
        for repeat in range(2 if op=="M" else 1):
            for vpn in range(addr//4096,(addr+size-1)//4096+1):
                out.write(str(vpn)+"\n")
```



캐시 크기와 LRU 히트율: 시드 2026의 합성 지역성 참조열 3000개.

23장 / 핵심 요약

완전한 VM: VAX/VMS

23장 | 제한된 하드웨어에 여러 기법을 결합한 실제 설계

하드웨어 제약과 해법

제약·문제	설계 선택	목적
512바이트의 작은 페이지	프로세스 공간을 P0·P1으로 나눠 각 테이블 관리	사용하지 않는 힙-스택 사이의 테이블 공간 감소
큰 사용자 페이지 테이블	커널 가상 공간에 두고 필요 시 테이블 페이지도 스왑	물리 메모리 사용 감소
reference bit 없음	프로세스별 FIFO + 전역 second-chance 리스트	단순한 정책을 보완하고 메모리 독점 완화
작은 페이지의 비효율적인 I/O	더티 페이지를 묶어서 기록	쓰기 요청 수 감소

교재의 VAX 주소 공간은 사용자 코드·힙의 P0, 스택의 P1, 보호된 커널의 시스템 공간으로 구성된다. 페이지 0을 접근 불가능하게 두면 NULL 역참조를 검출할 수 있다.

상주 집합과 두 번째 기회

프로세스마다 **RSS(상주 집합 크기)** 제한을 두고, 이를 넘으면 해당 FIFO의 오래된 페이지를 전역 clean/dirty 리스트로 옮긴다. 프레임이 다른 용도로 재사용되기 전에 다시 필요해지면 디스크에서 읽지 않고 되찾는다.

필요할 때만 수행하는 최적화

기법	미루는 작업	이득
Demand zero	실제로 접근할 때 프레임 확보·0 초기화	안 쓰는 페이지의 할당·초기화 비용 제거, 이전 데이터 노출 방지
Copy-on-write	두 주소 공간에 읽기 전용으로 공유한 후 쓰는 순간 복사	fork 후 exec처럼 복사할 필요가 사라지는 경우 낭비 감소

참조 비트가 없어도 잠시 접근을 금지하고 첫 접근의 트랩에서 권한을 복원해 사용 이력을 근사할 수 있다. 정확한 원래 권한은 따로 보관해야 하며, 너무 자주 반복하면 비용이 커진다.

23장 / 핵심 요약

Linux VM과 보호

23장 보충 | 성능·메모리 배치·보안을 함께 설계

기능	핵심
kmalloc / vmalloc	교재 모델에서 kmalloc은 물리적으로도 연속, vmalloc은 가상 주소상 연속이며 물리 배치는 분산 가능
다단계 페이지 테이블	첨부의 x86-64 예는 주소 인덱스 9+9+9+9비트와 offset 12비트인 4단계·48비트 모델
Huge pages	큰 페이지로 TLB 도달 범위 확대·테이블 감소. 내부 낭비·할당·스왑 비용과 절충
mmap	파일을 가상 주소에 매핑해 포인터 접근. 필요한 부분은 페이지 폴트 때 가져올 수 있음
활성·비활성 리스트	원본의 2Q 설명: 한 번 접근한 페이지와 재사용된 페이지를 구분해 스캔 오염 완화

32비트 3 GiB/1 GiB 분할, pdflush, 2Q 비율 등의 세부 값·명칭은 원본 시대의 설명이다. 핵심은 사용자/커널 보호, 백그라운드 기록, 재사용 여부를 반영한 회수다.

보호 장치가 막는 대상

장치	목적	한계·연결
NX / XD	스택 등 데이터 페이지에서 명령 실행 금지	기존 코드 조각을 재사용하는 ROP까지 모두 막지는 못함
ASLR / KASLR	사용자 / 커널 주소 배치를 무작위화	공격자가 코드·데이터 위치를 예측하기 어렵게 함
KPTI	사용자 테이블의 커널 매핑을 최소화하고 별도 커널 테이블 사용	일부 공격 경로 완화. 테이블 전환 비용이 있고 모든 추측 실행 공격을 해결하지는 못함

버퍼 오버플로는 할당 경계를 넘어 다른 메모리를 덮어쓰는 버그다. **Meltdown·Spectre**는 추측 실행의 캐시 등 미세구조 흔적을 통해 정보가 새는 문제를 보여 준다. 페이지 권한만으로 모든 정보 유출을 설명할 수 없다.

> **연결해서 기억하기**
> 페이지 테이블은 주소 변환뿐 아니라 공유·권한·COW·요구 할당·커널 격리의 기반이다. 실제 VM은 한 알고리즘이 아니라 여러 기법의 결합이다.

19 / 핵심 개념

핵심 공식과 검산

계산형 문제를 풀 때 단위·가정·추가 비용을 먼저 확인

대상	공식
반환·응답 시간	반환 = 완료 - 도착 / 응답 = 첫 실행 - 도착
주소 변환	VPN = floor(VA/P), offset = VA mod P PA = PFN×P + offset
페이지 테이블 크기	선형 테이블: (가상 주소 공간 크기/P) × PTE 크기
비트 수	$P=2^p$ 바이트 → offset p비트 a비트 VA → VPN a-p비트
TLB 도달 범위	엔트리 수 × 페이지 크기
Stride	stride = K/티켓; 최소 pass 선택; 실행 후 pass 증가

AMAT: 원본 보충의 개정식

$$AMAT = T_M + P_{miss} \times T_D$$

T_M은 기본 메모리 접근 시간, T_D는 미스 때 추가되는 저장 장치 접근 비용이다. P_{miss}는 백분율 대신 0-1 사이의 확률로 넣는다. 미스에도 기본 메모리 비용을 부담하는 모델이다.

조건: T_M=100 ns, T_D=10 ms	검산 결과
미스율 10% = 0.1	100 ns + 1 ms = **1.0001 ms**
히트율 99.9% → 미스율 0.001	100 ns + 10 µs = **10.1 µs**
둘의 비율	1.0001 ms / 10.1 µs **99.02배**

TLB 평균 시간의 단순 모델

TLB 조회 t, RAM 접근 m, 히트율 h, 선형 테이블·페이지 폴트 없음·캐시 효과 없음이면 **EAT = h(t+m)+(1-h)(t+2m)**. 다단계 n단계 미스에서는 t+(n+1)m을 사용한다. 문제의 조회 중첩·캐시 가정이 다르면 식도 바뀐다.

단위: 1 KiB=2^{10} B, 1 MiB=2^{20} B, 1 GiB=2^{30} B. 1 ms=1,000 µs=1,000,000 ns. 원본의 KB/MB 표기는 이진 단위 계산에서 KiB/MiB로 명확히 썼다.

20 / 핵심 개념

마지막 복습: 꼭 구분할 것

정의와 원인을 한 문장으로 설명할 수 있으면 핵심을 이해한 것

비교	구분 기준
프로그램 / 프로세스	저장된 코드 / 실행 상태를 가진 인스턴스
Ready / Blocked	CPU를 기다림 / I/O 등 사건을 기다림
fork / exec	새 프로세스 생성 / 현재 프로그램 이미지 교체
정책 / 메커니즘	선택 규칙 / 선택을 수행하는 구현
응답 / 반환 시간	첫 실행까지 / 완료까지
MLFQ / 비례 배분	관측한 행동으로 응답·반환 개선 / 정해진 CPU 몫 배분
세그멘테이션 / 페이징	논리 영역별 가변 크기 / 고정 크기 페이지
내부 / 외부 단편화	할당 안의 낭비 / 빈 공간의 분산
TLB 미스 / 페이지 폴트	주소 변환 캐시에 없음 / 페이지 상태·권한 처리가 필요
PTE valid / present	가상 공간에서 사용 가능한 매핑 / 지금 RAM에 있음
LRU / Clock	정확한 최근 사용 순서 / 참조 비트로 근사
Demand zero / COW	첫 사용 때 0 초기화 / 첫 쓰기 때 복사

스스로 확인할 네 가지

- 타이머 인터럽트가 없다면, 양보하지 않는 프로세스로부터 OS가 CPU를 회수할 수 있는가?
- TLB 미스인데 페이지가 RAM에 있는 경우와, 디스크에 있는 경우를 각각 설명할 수 있는가?
- 프레임을 늘리면 항상 페이지 폴트가 줄어드는가? FIFO와 LRU의 차이를 말할 수 있는가?
- fork 직후 exec할 자식의 메모리를 전부 복사하는 대신 COW를 쓰면 왜 유리한가?

각 장의 요약으로 연결 관계를 잡고, 문제 원문을 푼 뒤 같은 장의 번호별 해설로 확인한다. 이 통합본의 목차와 PDF 북마크를 이용해 이동할 수 있다.

부록 / 검산 기준과 출처

답의 기준과 재현 방법

> **문제는 첨부 PDF의 원문을 그대로 수록했다.**
> 정답·해설은 이 문서에서 작성한 예시 풀이이며, 교재 저자가 배포한 공식 정답집을 복제한 것이 아니다.

시뮬레이션 검산

공식 OSTEP Homework 저장소의 고정 커밋 afb36ca8ddbf81d847d18f6bd18a87f0a18667f2를 사용했다. 최초 155개와 추가 20개 설정을 실행했다. 의도적으로 제약을 어긴 입력의 오류도 확인했다. 무작위 그래프는 명시한 시드·모형으로 생성했다.

교재 저자 공식 저장소 · 검산에 사용한 버전

숫자 답을 읽을 때

단위 틱은 시뮬레이터의 모델 시간이다. 실제 CPU의 ns·ms 측정값과 구분한다. 무작위 문제는 시드뿐 아니라 기본값·프로그램 버전·난수 소비 순서에도 의존한다. 따로 예시 조건을 정한 자유형 문제에는 그 조건을 표시했다.

실측·코드 문제

시스템 콜·문맥 교환·TLB·free/pmap/vmstat 수치는 환경에 따라 달라진다. 이 책은 실제 사용자 Linux 장비의 측정 결과를 만들어 넣지 않았다. 코드와 관측 지표, 해석, 결과를 확인할 명령을 제시했다. 오류 예시는 의도적으로 잘못된 코드를 표시했다.

번역·표기 차이

위치	해설에 반영한 차이
04장 문제 4	4:200은 CPU 비율 표기의 오기. 4:100과 결과가 같음을 확인
14장 문제 5	data[l00]은 data[100]
15장 문제 3	영문 원문은 bound가 아닌 base의 최댓값을 질문
22장 문제 4	원본 보충 256쪽에서 LRU와 RAND 비교 방향을 교정
23장	첨부 판본에 독립 숙제 없음; Linux 보충은 요약에 포함

참고한 1차 자료

OSTEP 15장 영문 원문(문제 3 대조) · fork(2) · wait(2) · malloc/realloc(3)

procps-ng free(1) · procps-ng vmstat(8) · Valgrind Lackey 공식 매뉴얼